

基于Kinect V2的植物表型特征 提取与重构



冀杰



西南大学 工程技术学院



2026.04.22

01

MATLAB/Kinect基本操作

01 MATLAB/Kinect基本操作

1. Detect the Kinect V2 Devices:

Typically, each camera or image device in the Image Acquisition Toolbox™ has one DeviceID. Because the Kinect® for Windows® camera has two separate sensors, the color sensor and the depth sensor, the toolbox lists two DeviceIDs. Use `imaqhwinfo` on the adaptor to display the two device IDs.

```
info = imaqhwinfo('kinect');  
info
```

```
info =
```

```
AdaptorDllName: '<matlabroot>\toolbox\imaq\supportpackages\kinectruntime\adaptor  
               \win64\mwkinectimaq.dll'
```

```
AdaptorDllVersion: '5.0 (R2016a)'
```

```
AdaptorName: 'kinect'
```

```
DeviceIDs: {[1] [2]}
```

```
DeviceInfo: [1x2 struct]
```

01 MATLAB/Kinect基本操作

1. Detect the Kinect V2 Devices:

If you look at each device, you can see that they represent the color sensor and the depth sensor. The following shows the color sensor.

The following shows the depth sensor, which is Device 2

You can use multiple Kinect devices together, but only one Kinect V2 at a time. Multiple Kinect sensors are enumerated as DeviceIDs [1] [2] [3] [4] and so on. For example, if you had two Kinect devices, a Kinect V1 and a Kinect V2, the first one would have Kinect Color Sensor with DeviceID 1 and Kinect Depth Sensor with DeviceID 2 and the second Kinect device would have Kinect V2 Color Sensor with DeviceID 3 and Kinect V2 Depth Sensor with DeviceID 4.

```
info.DeviceInfo(1)
```

```
ans =
```

```
DefaultFormat: 'RGB_1920x1080'  
DeviceFileSupported: 0  
DeviceName: 'Kinect V2 Color Sensor'  
DeviceID: 1  
VideoInputConstructor: 'videoinput('kinect', 1)'  
VideoDeviceConstructor: 'imaq.VideoDevice('kinect', 1)'  
SupportedFormats: 'RGB_1920x1080'
```

```
info.DeviceInfo(2)
```

```
ans =
```

```
DefaultFormat: 'Depth_512x424'  
DeviceFileSupported: 0  
DeviceName: 'Kinect V2 Depth Sensor'  
DeviceID: 2  
VideoInputConstructor: 'videoinput('kinect', 2)'  
VideoDeviceConstructor: 'imaq.VideoDevice('kinect', 2)'  
SupportedFormats: 'Depth_512x424'
```

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

In Detect the Kinect V2 Devices, you see that the two sensors on the Kinect® for Windows® device are represented by two device IDs, one for the color sensor and one of the depth sensor. In that example, Device 1 is the color sensor and Device 2 is the depth sensor. This example shows how to create a videoinput object for the color sensor to acquire RGB images and then for the depth sensor to acquire body data.

1. Create the videoinput object for the color sensor. DeviceID 1 is used for the color sensor.

```
vid = videoinput('kinect',1);
```

Note that you do not need to provide the video format as you do for a Kinect V1 device, since only one format is used in Kinect V2 devices (RGB_1920x1080).

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

2. Look at the device-specific properties on the source device, which is the color sensor on the Kinect V2 camera.

The output shows that the color sensor has a set of device-specific properties. These properties are read-only for Kinect V2. You can set properties on Kinect V1 devices, but not on Kinect V2 devices. The Kinect V2 device can change the properties, depending on conditions.

Device-Specific Property – Color Sensor	Description
ExposureTime	Indicates the exposure time in increments of 1/10,000 of a second.
FrameInterval	Indicates the frame interval in units of 1/1,000,000 of a second.
Gain	Indicates a multiplier for the RGB color values.
Gamma	Indicates gamma measurement.

```
src = getselectedsource(vid);
```

```
src
Display Summary for Video Source Object:
General Settings:
Parent = [1x1 videoinput]
Selected = on
SourceName = Kinect V2 Color Source
Tag =
Type = videosource
```

```
Device Specific Properties:
ExposureTime = 4000
FrameInterval = 333333
Gain = 1
Gamma = 2.2
```

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

3. Preview the color stream by calling preview on the color sensor object you created.

```
preview(vid);
```

When you are done previewing, close the preview window.

```
closepreview(vid);
```

4. Create the videoinput object for the depth sensor. Note that a second object is created (vid2), and DeviceID 2 is used for the depth sensor.

```
vid2 = videoinput('kinect', 2);
```

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

5. Look at the device-specific properties on the source device, which is the depth sensor on the Kinect V2 camera.

The output shows that the depth sensor has one device-specific property associated with body tracking. This property is specific to the depth sensor.

6. Collect body metadata by turning on body tracking, which is off by default.

```
src.EnableBodyTracking = 'on';
```

7. Start the second videoinput object (the depth stream).

```
start(vid2);
```

```
src = getselectedsource(vid2);
```

```
src
```

Display Summary for Video Source Object:

General Settings:

Parent = [1x1 videoinput]

Selected = on

SourceName = Kinect V2 Depth Source

Tag =

Type = videosource

Device Specific Properties:

EnableBodyTracking = off

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

8. Access body tracking data as metadata on the depth stream using `getdata`. The function returns:.

- Frames of size 512x424 in mono13 format and uint16 data type
- Time stamps
- Metadata

9. Look at any individual property by drilling into the metadata.
For example, look at the `IsBodyTracked` property.

```
metaData.IsBodyTracked
```

```
ans = 1    0    0    0    0    0
```

In this case the data shows that of the six possible bodies, there is one body being tracked and it is in the first position. If you have multiple bodies, this property is useful to confirm which ones are being tracked.

```
% Get the data on the object.
```

```
[frame, ts, metaData] = getdata(vid2);
```

```
% Look at the metadata to see the parameters in  
the body data.
```

```
metaData
```

```
metaData =
```

```
11x1 struct array with fields:
```

```
IsBodyTracked: [1x6 logical]
```

```
BodyTrackingID: [1x6 double]
```

```
BodyIndexFrame: [424x512 double]
```

```
ColorJointIndices: [25x2x6 double]
```

```
DepthJointIndices: [25x2x6 double]
```

```
HandLeftState: [1x6 double]
```

```
HandRightState: [1x6 double]
```

```
HandLeftConfidence: [1x6 double]
```

```
HandRightConfidence: [1x6 double]
```

```
JointTrackingStates: [25x6 double]
```

```
JointPositions: [25x3x6 double]
```

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

10. Get the joint locations for the first body using the JointPositions property. Since this is the body in position 1, the index uses 1.

The columns represent the X, Y, and Z coordinates in meters of the 25 points on body 1.

11. Optionally view the segmentation data as an image using the BodyIndexFrame property.

% View the segmentation data as an image.

`imagesc(metaDataDepth.BodyIndexFrame);`

% Set the color map to jet to color code the people detected.

`colormap(jet);`

`metaData.JointPositions(:, :, 1)`

`ans =`

-0.1408	-0.3257	2.1674
-0.1408	-0.2257	2.1674
-0.1368	-0.0098	2.2594
-0.1324	0.1963	2.3447
-0.3024	-0.0058	2.2574
-0.3622	-0.3361	2.1641
-0.3843	-0.6279	1.9877
-0.4043	-0.6779	1.9877
0.0301	-0.0125	2.2603
0.2364	0.2775	2.2117
0.3775	0.5872	2.2022
0.4075	0.6372	2.2022
-0.2532	-0.4392	2.0742
-0.1869	-0.8425	1.8432
-0.1869	-1.2941	1.8432
-0.1969	-1.3541	1.8432
-0.0360	-0.4436	2.0771
0.0382	-0.8350	1.8286
0.1096	-1.2114	1.5896
0.1196	-1.2514	1.5896
0.2969	1.2541	1.2432
0.1360	0.5436	1.1771
0.1382	0.7350	1.5286
0.2096	1.2114	1.3896
0.0196	1.1514	1.4896

01 MATLAB/Kinect基本操作

2. Acquire Image and Body Data Using Kinect V2:

Joint Positions

The EnableBodyTracking property indicates whether body metadata is collected. When set to on, this list displays the order of the joints returned by the Kinect V2 adaptor in the JointPositions property.

```
SpineBase = 1;  
SpineMid = 2;  
Neck = 3;  
Head = 4;  
ShoulderLeft = 5;  
ElbowLeft = 6;  
WristLeft = 7;  
HandLeft = 8;  
ShoulderRight = 9;  
ElbowRight = 10;  
WristRight = 11;  
HandRight = 12;  
HipLeft = 13;  
KneeLeft = 14;  
AnkleLeft = 15;  
FootLeft = 16;  
HipRight = 17;  
KneeRight = 18;  
AnkleRight = 19;  
FootRight = 20;  
SpineShoulder = 21;  
HandTipLeft = 22;  
ThumbLeft = 23;  
HandTipRight = 24;  
ThumbRight = 25;
```

01 MATLAB/Kinect基本操作

3. Acquire Data from Kinect V2 Color and Depth Devices Simultaneously:

You can synchronize the data from the Kinect® for Windows® V2 color stream and the depth stream using software manual triggering.

This example shows the synchronization method used to manually trigger both objects.

1. Create the objects for the color and depth sensors. Device 1 is the color sensor and Device 2 is the depth sensor.

```
vid = videoinput('kinect',1);  
vid2 = videoinput('kinect',2);
```

2. Get the source properties for the depth device.

```
srcDepth = getselectedsource(vid2);
```

01 MATLAB/Kinect基本操作

3. Acquire Data from Kinect V2 Color and Depth Devices Simultaneously:

3. Set the frames per trigger for both devices to 1.

```
vid.FramesPerTrigger = 1;  
vid2.FramesPerTrigger = 1;
```

4. Set the trigger repeat for both devices to 200, in order to acquire 201 frames from both the color sensor and the depth sensor.

```
vid.TriggerRepeat = 200;  
vid2.TriggerRepeat = 200;
```

5. Configure the camera for manual triggering for both sensors.

```
triggerconfig([vid vid2],'manual');
```

01 MATLAB/Kinect基本操作

3. Acquire Data from Kinect V2 Color and Depth Devices Simultaneously:

6. Start both video objects.

```
start([vid vid2]);
```

7. Trigger the devices, then get the acquired data.

```
% Trigger 200 times to get the frames.
```

```
for i = 1:200
```

```
    % Trigger both objects.
```

```
    trigger([vid vid2])
```

```
    % Get the acquired frames and metadata.
```

```
    [imgColor, ts_color, metaData_Color] = getdata(vid);
```

```
    [imgDepth, ts_depth, metaData_Depth] = getdata(vid2);
```

```
end
```

```
delete([vid vid2]);
```

02

MATLAB/Kinect深度图像获取与分析

02 MATLAB/Kinect深度图像获取与分析

1. Preview Color and Depth Streams Simultaneously from Kinect for Windows V2

1) introduction

This example shows how to preview color and depth streams using the Kinect® for Windows® V2.

Requirements

- MATLAB® and Image Acquisition Toolbox™
- Kinect for Windows V2 sensor
- Minimum PC configuration: Windows 10 64-bit

2) Identify Kinect for Windows V2

The `imaqhwinfo("kinect")` command provides information on the Kinect's color and depth devices. Since the Kinect has two sensors, a color and a depth sensor, it is enumerated as two devices.

```
kinectDeviceInfo = imaqhwinfo("kinect");
```

```
kinectDeviceInfo.DeviceIDs
```

```
ans=1×2 cell array
```

```
{[1]} {[2]}
```


02 MATLAB/Kinect深度图像获取与分析

1. Preview Color and Depth Streams Simultaneously from Kinect for Windows V2

1) introduction

This example shows how to preview color and depth streams using the Kinect® for Windows® V2.

Requirements

- MATLAB® and Image Acquisition Toolbox™
- Kinect for Windows V2 sensor
- Minimum PC configuration: Windows 10 64-bit

2) Identify Kinect for Windows V2

The `imaqhwinfo("kinect")` command provides information on the Kinect's color and depth devices. Since the Kinect has two sensors, a color and a depth sensor, it is enumerated as two devices.

```
kinectDeviceInfo = imaqhwinfo("kinect");
```

```
kinectDeviceInfo.DeviceIDs
```

```
ans=1×2 cell array
```

```
    {[1]}    {[2]}
```

Only one Kinect for Windows V2 can be used per computer. However, you can use the Kinect V1 and Kinect V2 together in the same MATLAB session. The devices will be enumerated as devices IDs [1],[2] (Kinect V1) and [3],[4] (Kinect V2).

02 MATLAB/Kinect深度图像获取与分析

1. Preview Color and Depth Streams Simultaneously from Kinect for Windows V2

3) Obtain Kinect V2 Device Information

Information on a specific device can be obtained by simply indexing into the device information structure array.

Get Kinect color device information.

```
colorDevice = kinectDeviceInfo.DeviceInfo(1)
```

colorDevice = struct with fields:

DefaultFormat: 'BGR_1920x1080'

DeviceFileSupported: 0

DeviceName: 'Kinect V2 Color Sensor'

DeviceID: 1

VideoInputConstructor: 'videoinput('kinect', 1)'

VideoDeviceConstructor: 'imaq.VideoDevice('kinect', 1)'

SupportedFormats: {'BGR_1920x1080'}

Get Kinect depth device information.

```
depthDevice = kinectDeviceInfo.DeviceInfo(2)
```

depthDevice = struct with fields:

DefaultFormat: 'Depth_512x424'

DeviceFileSupported: 0

DeviceName: 'Kinect V2 Depth Sensor'

DeviceID: 2

VideoInputConstructor: 'videoinput('kinect', 2)'

VideoDeviceConstructor: 'imaq.VideoDevice('kinect', 2)'

SupportedFormats: {'Depth_512x424'}

02 MATLAB/Kinect深度图像获取与分析

1. Preview Color and Depth Streams Simultaneously from Kinect for Windows V2

4) Create Color and Depth videoinput Objects

A videoinput object represents the connection between MATLAB and an image acquisition device.

Create a color videoinput object.

```
colorVid = videoinput("kinect",1)
```

Create a depth videoinput object.

```
depthVid = videoinput("kinect",2)
```

5) Preview Both Color and Depth Objects

Preview both videoinput objects.

```
preview([colorVid depthVid]);
```

6) Clear videoinput Objects

```
delete(colorVid);
```

```
delete(depthVid);
```

02 MATLAB/Kinect深度图像获取与分析

2. Plot Color Point Cloud from Kinect for Windows V2:

This example shows how to plot a color point cloud using the Kinect® for Windows® V2. Requirements:

- MATLAB®, Image Acquisition Toolbox™, and Computer Vision Toolbox™
- Kinect for Windows V2 sensor
- Minimum PC configuration: Windows 10 64-bit

1) Create System Objects for Kinect V2 Device

The Kinect camera has separate sensors for color and depth. Create separate system objects for the color sensor and depth sensor.

```
colorDevice = imaq.VideoDevice("kinect",1)
```

colorDevice = imaq.VideoDevice with properties:

Device: 'Kinect V2 Color Sensor (kinect-1)'

VideoFormat: 'BGR_1920x1080'

ROI: [1 1 1920 1080]

ReturnedColorSpace: 'rgb'

ReturnedDataType: 'uint8'

ReadAllFrames: 'off'

DeviceProperties: [1×1 imaq.internal.DeviceProperties]

02 MATLAB/Kinect深度图像获取与分析

2. Plot Color Point Cloud from Kinect for Windows V2:

1) Create System Objects for Kinect V2 Device

```
depthDevice = imaq.VideoDevice("kinect",2)
```

depthDevice = imaq.VideoDevice with properties:

Device: 'Kinect V2 Depth Sensor (kinect-2)'

VideoFormat: 'Depth_512x424'

ROI: [1 1 512 424]

ReturnedColorSpace: 'grayscale'

ReturnedDataType: 'uint16'

ReadAllFrames: 'off'

DeviceProperties: [1×1 imaq.internal.DeviceProperties]

Initialize the sensors.

```
colorDevice();  
depthDevice();
```

Grab one frame from each of the sensors.

```
colorImage = colorDevice();  
depthImage = depthDevice();
```

02 MATLAB/Kinect深度图像获取与分析

2. Plot Color Point Cloud from Kinect for Windows V2:

2) Extract Point Cloud

depthDevice must be a Kinect depth videoinput object or a Kinect depth imaq.VideoDevice object.
depthimage must be uint16 and colorImage must be uint8.

```
ptCloud = pcfromkinect(depthDevice,depthImage,colorImage);
```

The Kinect for Windows system, designed for gaming, produces depthImage and colorImage as mirror images of the scene. The returned point cloud is corrected to match the actual scene.

Initialize a player to visualize the 3-D point cloud data. The axis is set appropriately to visualize the point cloud from Kinect.

```
player = pcplayer(ptCloud.XLimits,ptCloud.YLimits,ptCloud.ZLimits,"VerticalAxis","y","VerticalAxisDir","down");  
xlabel(player.Axes,"X (m)");  
ylabel(player.Axes,"Y (m)");  
zlabel(player.Axes,"Z (m)");
```


02 MATLAB/Kinect深度图像获取与分析

2. Plot Color Point Cloud from Kinect for Windows V2:

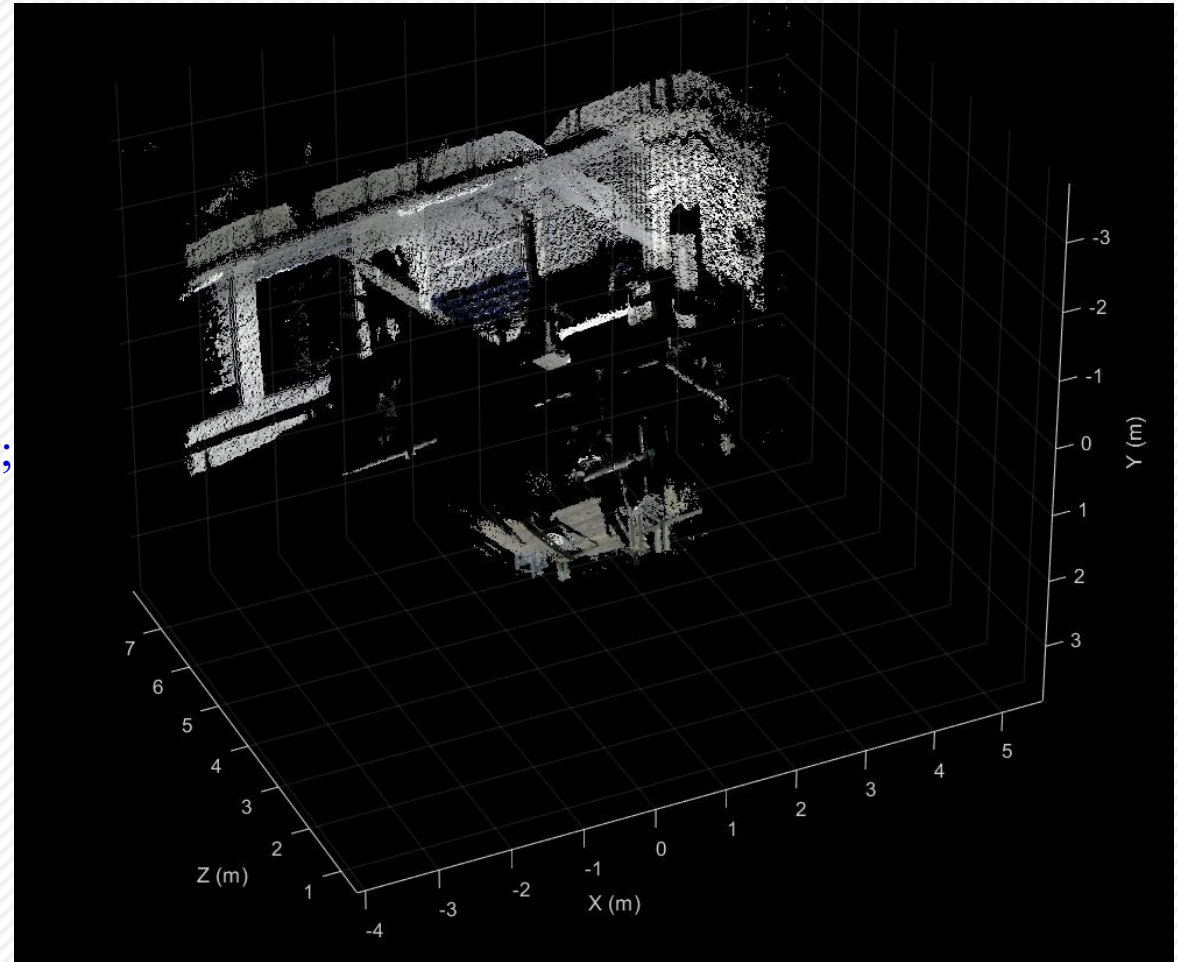
2) Extract Point Cloud

Acquire and view Kinect point cloud data.

```
while isOpen(player)
    colorImage = colorDevice();
    depthImage = depthDevice();
    ptCloud = pcfromkinect(depthDevice,depthImage,colorImage);
    view(player,ptCloud);
end
```

Release the Devices

```
release(colorDevice);
release(depthDevice);
```



02 MATLAB/Kinect深度图像获取与分析

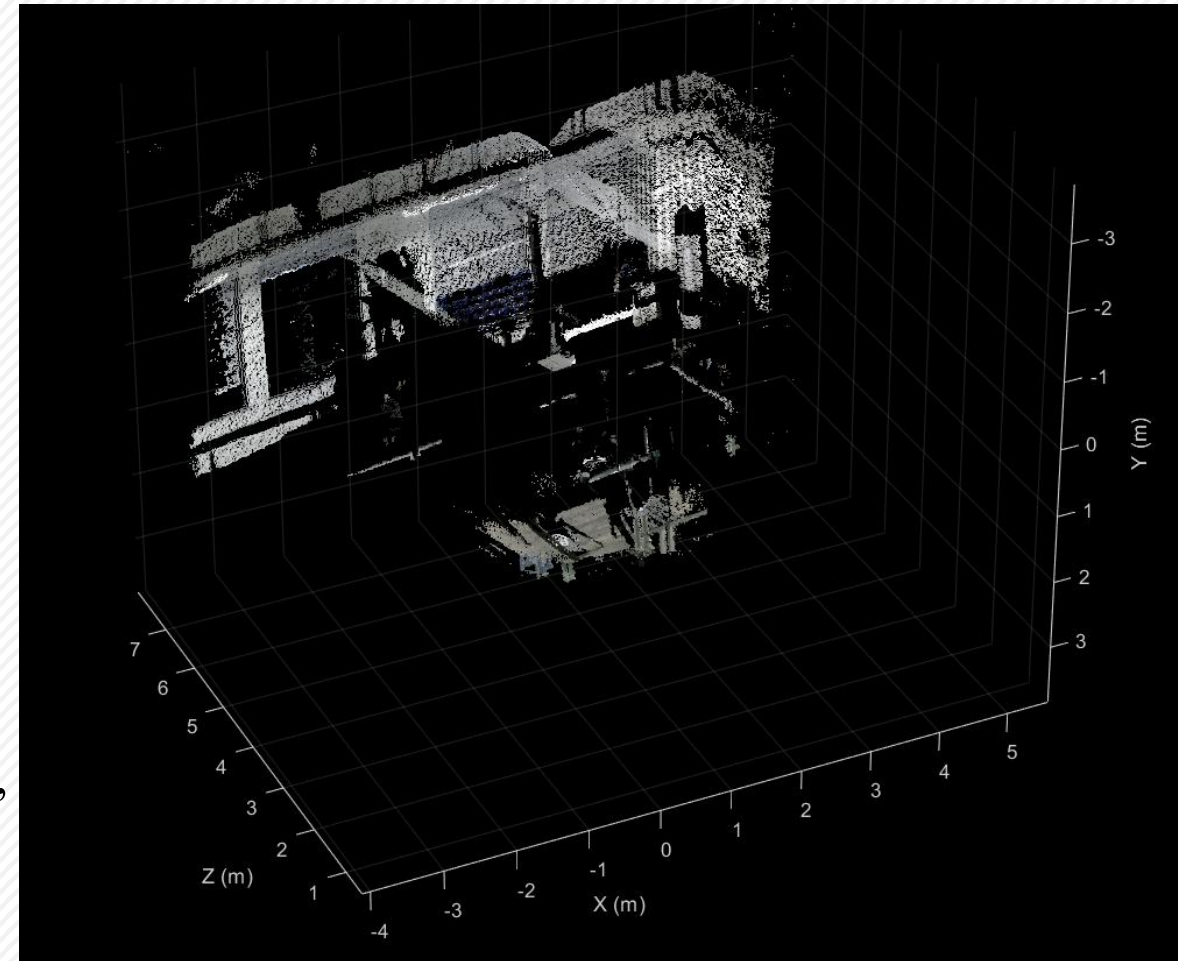
2. Plot Color Point Cloud from Kinect for Windows V2:

2) Extract Point Cloud

The origin of a right-handed world coordinate system is at the center of the camera. The X axis of the coordinate system is pointing to the right, the Y axis is pointing downward, and the Z axis is pointing away from the camera.

Since the Kinect depth camera has limited range, some pixels in the depth image do not have corresponding 3-D coordinates. The values for those pixels are set to NaN in the Location property of ptCloud.

Since Kinect was designed for gaming, the original images, colorImage and depthImage, from Kinect are mirror images of the scene. The returned point cloud is corrected to match the actual scene.



02 MATLAB/Kinect深度图像获取与分析

3. View Skeletal Data from Kinect for Windows V2

This example shows how to view an RGB image from the Kinect® for Windows® V2 with the skeleton joint locations overlaid on the image. Requirements

- MATLAB® and Image Acquisition Toolbox™
- Kinect for Windows V2 sensor
- Minimum PC configuration: Windows 10 64-bit with a dedicated USB3 controller

1) Set Up Kinect V2 for Color and Depth Acquisition

Create color and depth videoinput objects.

<code>colorVid = videoinput("kinect",1)</code>	Summary of Video Input Object Using 'Kinect V2 Color Sensor'. Acquisition Source(s): Kinect V2 Color Source is available. Acquisition Parameters: 'Kinect V2 Color Source' is the current selected source. 10 frames per trigger using the selected source. 'BGR_1920x1080' video data to be logged upon START. Grabbing first of every 1 frame(s). Log data to 'memory' on trigger.
Create a depth videoinput object.	
<code>depthVid = videoinput("kinect",2)</code>	Trigger Parameters: 1 'immediate' trigger(s) on START. Status: Waiting for START. 0 frames acquired since starting. 0 frames available for GETDATA.

02 MATLAB/Kinect深度图像获取与分析

3. View Skeletal Data from Kinect for Windows V2

This example shows how to view an RGB image from the Kinect® for Windows® V2 with the skeleton joint locations overlaid on the image. Requirements

- MATLAB® and Image Acquisition Toolbox™
- Kinect for Windows V2 sensor
- Minimum PC configuration: Windows 10 64-bit with a dedicated USB3 controller

1) Set Up Kinect V2 for Color and Depth Acquisition

Create color and depth videoinput objects.

```
colorVid = videoinput("kinect",1)
```

Create a depth videoinput object.

```
depthVid = videoinput("kinect",2)
```

Summary of Video Input Object Using 'Kinect V2 Depth Sensor'.

Acquisition Source(s): Kinect V2 Depth Source is available.

Acquisition Parameters: 'Kinect V2 Depth Source' is the current selected source.

10 frames per trigger using the selected source.

'Depth_512x424' video data to be logged upon START.

Grabbing first of every 1 frame(s).

Log data to 'memory' on trigger.

Trigger Parameters: 1 'immediate' trigger(s) on START.

Status: Waiting for START.

0 frames acquired since starting.

0 frames available for GETDATA.

02 MATLAB/Kinect深度图像获取与分析

3. View Skeletal Data from Kinect for Windows V2

Look at the device-specific properties on the depth source device, which is the depth sensor on the Kinect V2.

Set 'EnableBodyTracking' to on, so that the depth sensor returns body tracking metadata along with the depth frame.

```
depthSource = getselectedsource(depthVid);
```

```
depthSource.EnableBodyTracking = "on";
```

Acquire 100 color and depth frames.

```
framesPerTrig = 100;
```

```
colorVid.FramesPerTrigger = framesPerTrig;
```

```
depthVid.FramesPerTrigger = framesPerTrig;
```

Start the depth and color acquisition objects. This begins acquisition, but does not start logging of acquired data.

```
pause(5);
```

```
start([depthVid colorVid]);
```

02 MATLAB/Kinect深度图像获取与分析

3. View Skeletal Data from Kinect for Windows V2

2) Access Image and Skeletal Data

Get images and metadata from the color and depth device objects.

```
[colorImg] = getdata(colorVid);
```

```
[~,~,metadata] = getdata(depthVid);
```

This is the order of joints returned by the Kinect.

Create a skeleton connection map to link the joints.

```
SpineBase = 1;  
SpineMid = 2;  
Neck = 3;  
Head = 4;  
ShoulderLeft = 5;  
ElbowLeft = 6;  
WristLeft = 7;  
HandLeft = 8;  
ShoulderRight = 9;  
ElbowRight = 10;  
WristRight = 11;  
HandRight = 12;  
HipLeft = 13;  
KneeLeft = 14;  
AnkleLeft = 15;  
FootLeft = 16;  
HipRight = 17;  
KneeRight = 18;  
AnkleRight = 19;  
FootRight = 20;  
SpineShoulder = 21;  
HandTipLeft = 22;  
ThumbLeft = 23;  
HandTipRight = 24;  
ThumbRight = 25;
```

```
SkeletonConnectionMap = [  
    4 3]; % Neck  
    3 21]; % Head  
    21 2]; % Right Leg  
    2 1];  
    21 9];  
    9 10]; % Hip  
    10 11];  
    11 12]; % Left Leg  
    12 24];  
    12 25];  
    21 5]; % Spine  
    5 6];  
    6 7]; % Left Hand  
    7 8];  
    8 22];  
    8 23];  
    1 17];  
    17 18];  
    18 19]; % Right Hand  
    19 20];  
    1 13];  
    13 14];  
    14 15];  
    15 16];  
];
```

02 MATLAB/Kinect深度图像获取与分析

3. View Skeletal Data from Kinect for Windows V2

Extract the 90th frame and tracked body information.

```
imageFrame = framesPerTrig-10;
```

```
imageframeMetadata = metadata(imageFrame);
```

Find the indexes of the tracked bodies.

```
anyBodiesTracked = any(imageframeMetadata.IsBodyTracked ~= 0);
```

```
trackedBodies = find(imageframeMetadata.IsBodyTracked);
```

Find number of skeletons tracked.

```
nBodies = length(trackedBodies);
```

Get the joint indices of the tracked bodies with respect to the color image.

```
colorJointIndices = imageframeMetadata.ColorJointIndices(:, :, trackedBodies);
```

Extract the 90th color frame.

```
colorImage = colorImg(:, :, :, imageFrame);
```


02 MATLAB/Kinect深度图像获取与分析

3. View Skeletal Data from Kinect for Windows V2

2) View RGB Image with Skeletal Overlay

Marker colors for up to six bodies.

```
colors = ["r";"g";"b";"c";"y";"m"];
```

Display the RGB image.

```
imshow(colorImage);
```

Overlay the skeleton on this RGB frame.

```
for i = 1:24
```

```
    for body = 1:nBodies
```

```
        X1 = [colorJointIndices(SkeletonConnectionMap(i,1),1,body) colorJointIndices(SkeletonConnectionMap(i,2),1,body)];
```

```
        Y1 = [colorJointIndices(SkeletonConnectionMap(i,1),2,body) colorJointIndices(SkeletonConnectionMap(i,2),2,body)];
```

```
        line(X1,Y1,"LineWidth",1.5,"LineStyle","-","Marker","+","Color",colors(body));
```

```
    end
```

```
    hold on;
```

```
end
```

```
hold off;
```



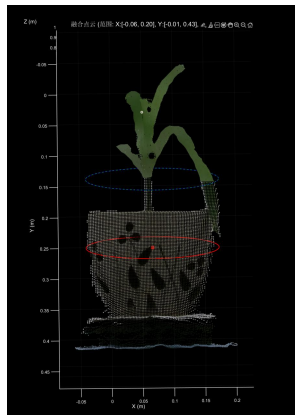
03 基于MATLAB/Kinect的表征三维建模

1. 基于点云及图像的植物三维建模方法



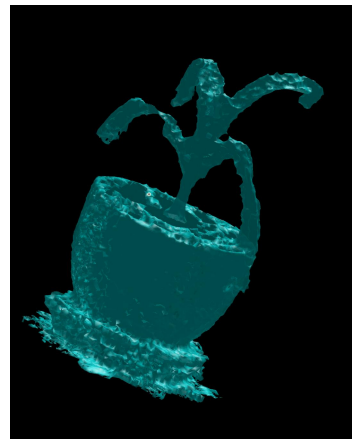
点云获取

使用Kinect V2.0获得彩色和深度数据



数据融合

三维点云数据与RGB色彩数据进行融合



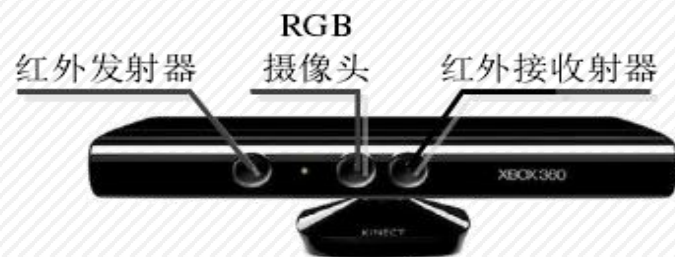
模型建立

多角度图像及点云进行拼接，建立三维模型



03 基于MATLAB/Kinect的表征三维建模

2. Kinect软硬件准备



Kinect V2

任务目标：玉米植株的表面三维点云模型的建立

硬件器材：Kinect V2相机；玉米幼苗；花盆；可旋转平台

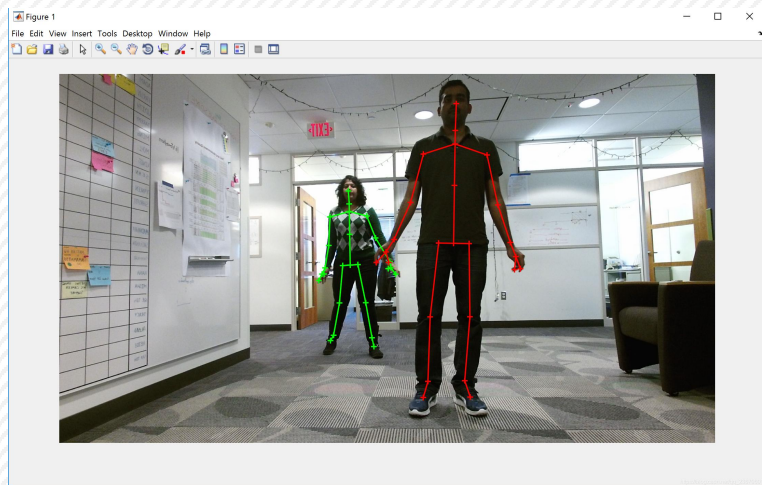
深度空间：用“深度”信息（即每个像素点距离相机的距离）来描述三维场景的空间。

色彩空间：用数学模型来表示颜色的方法。

空间点云：在三维空间中的点的集合，每个点通常有XYZ坐标，可以加上颜色、强度等属性

mat文件：MATLAB专用的数据存储格式，能高效存储变量、数组、结构体

点云文件：记录了每个点的空间坐标（以及可能的颜色、法向量等附加信息），便于3D可视化和处理。相较于mat文件结构更简单。

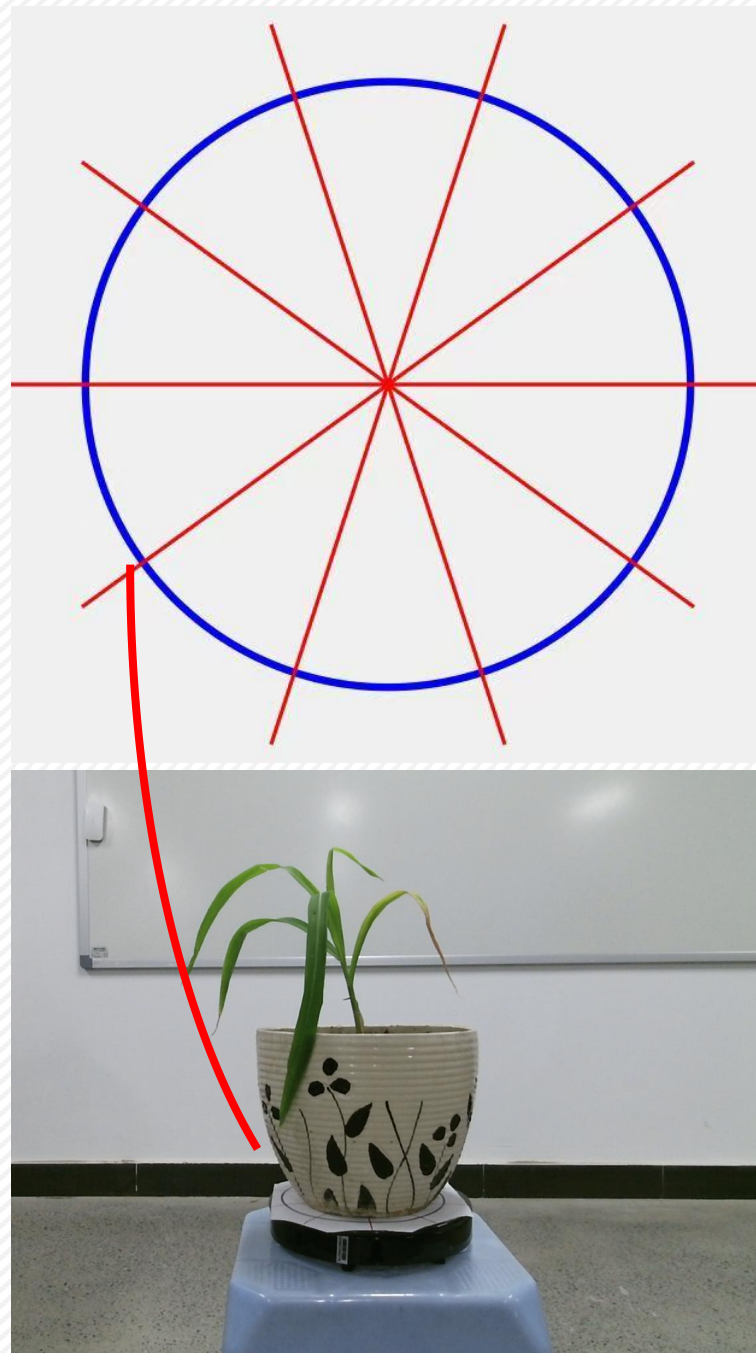


03 基于MATLAB/Kinect的表征三维建模

3. 数据获取场景布置

准备工作--获取原始数据

- ①固定相机、旋转平台、支撑座。
- ②设计10等分的圆形纸张，将其居中放置在旋转台上，花盆同时居中放置在旋转台上。
- ③根据辐射线的位置当旋转台每次旋转 36° ，旋转台停止，相机拍摄一次。
- ④如此循环拍摄，得到10组原始数据。



03 基于MATLAB/Kinect的表征三维建模

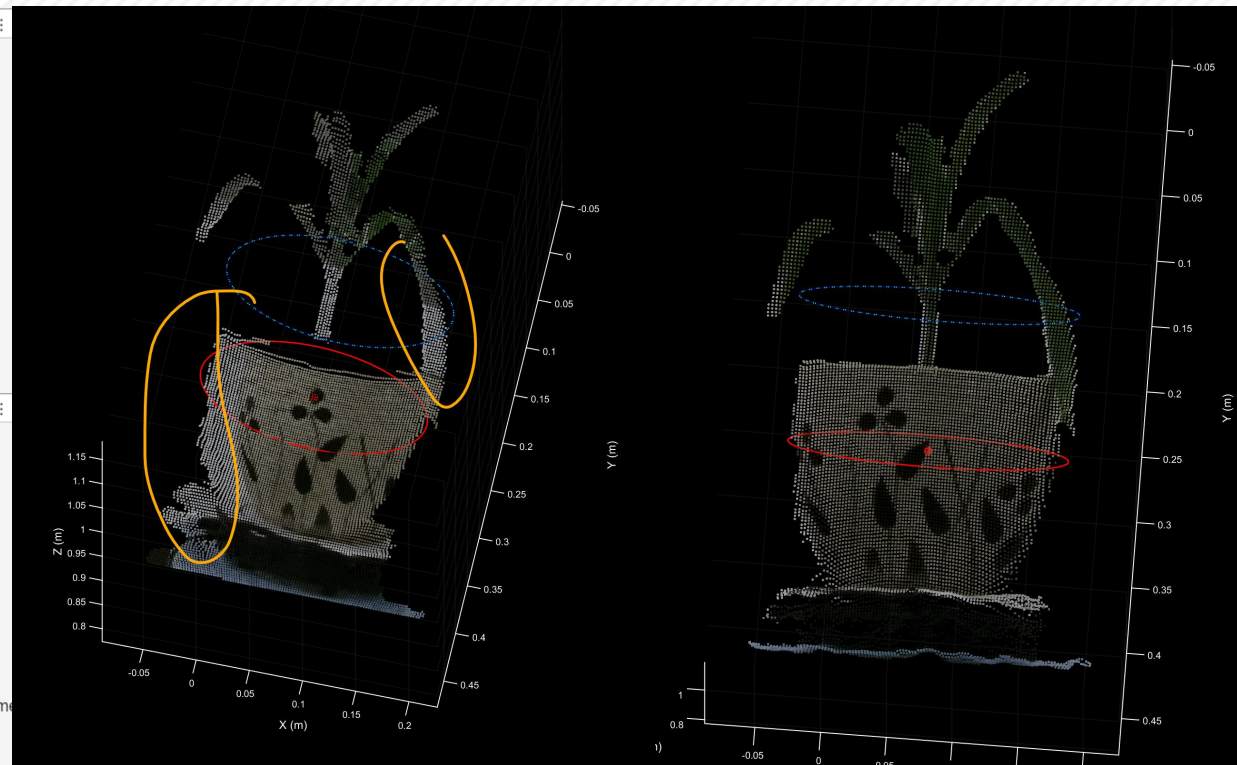
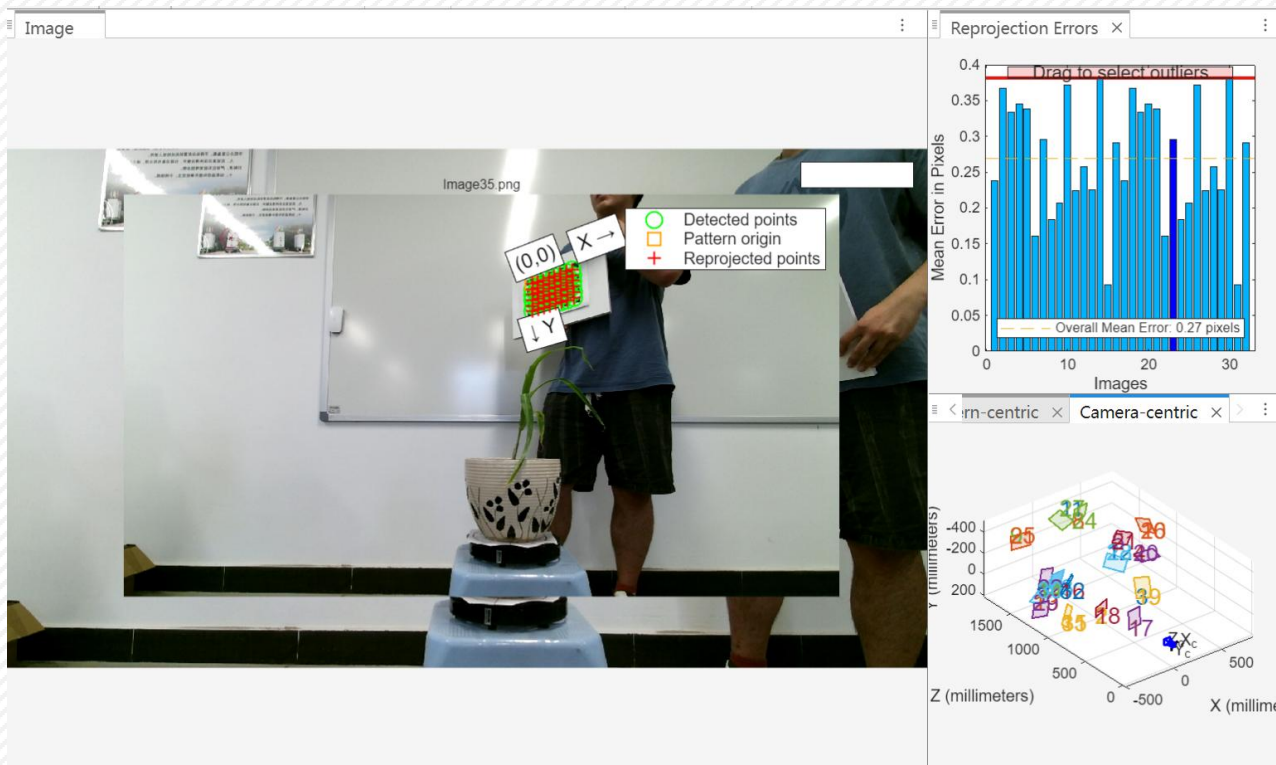
4. 相机标定及内外参确定

1.matlab标定内参:

使用方格盘进行标定获得相机内参

2. 粗细标定外参

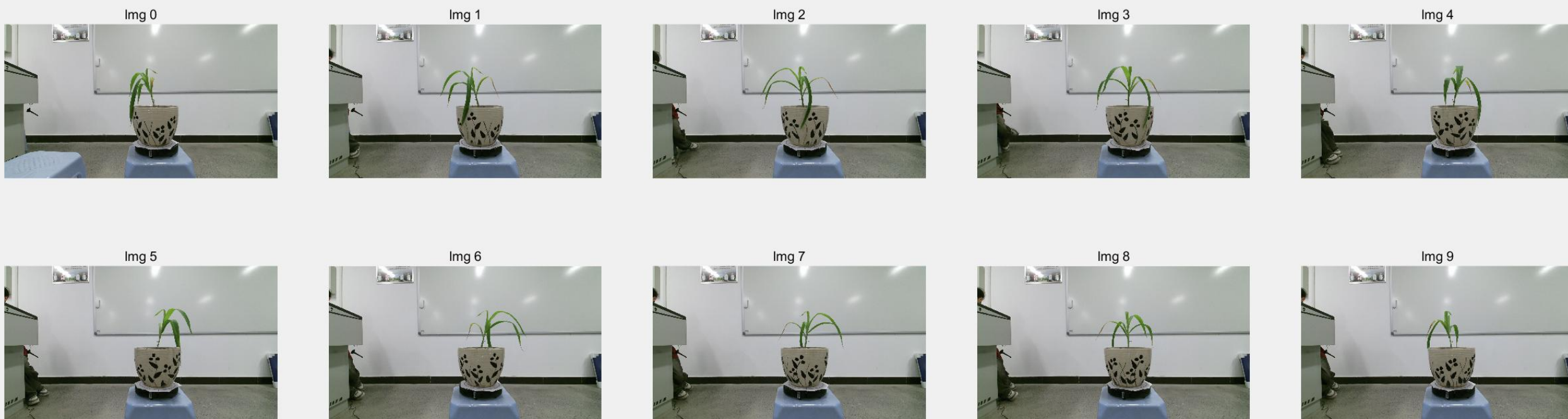
通过测量两个相机的物理位置粗标定外参
根据10组样本细标定求平均值



03 基于MATLAB/Kinect的表征三维建模

5. 数据获取场景布置及数据获取

将Kinect V2相机固定在一个三脚架上，相机摄像头正对花盆，且花盆在视野中相对位置保持不变，花盆放在一个小型转台上，转台放置在一个凳子上以保证花盆在相机视野里；花盆的圆心与转台圆心尽量保持重合，在转台上放置了一个被平均划分为十等份的纸张也尽量与圆台中心重合，通过控制转台进行低速转动，每使转台转动36度即纸张的十分之一时，获取一张深度图像以及与其对应的彩色图像。



03 基于MATLAB/Kinect的表征三维建模

6. 点云及图像数据提取

1. 提取数据

深度相机内参获得点云坐标

根据既定深度范围和宽度范围筛除

中值滤波进行深度去噪

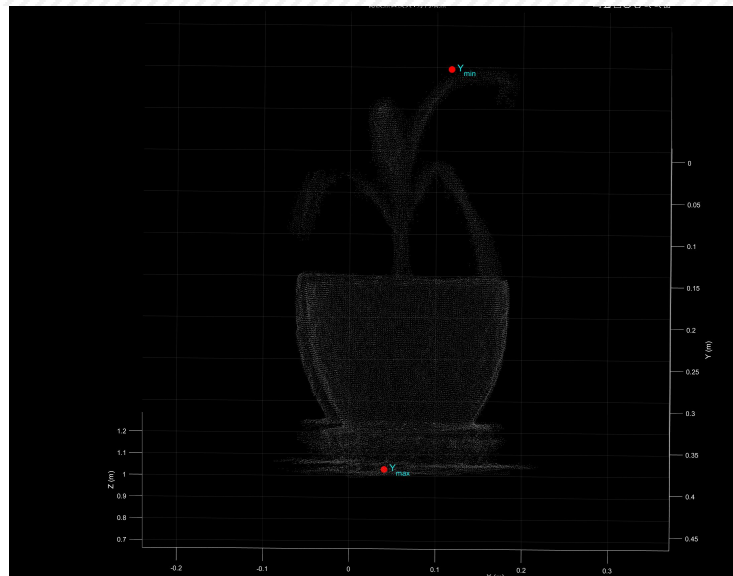
利用插值将彩色图像和点云坐标融合（分辨率不一样）

去除无色点

2. 自适应高度范围

对滤波获得有效高度范围，根据植株生长阶段设定后续处理标准高度

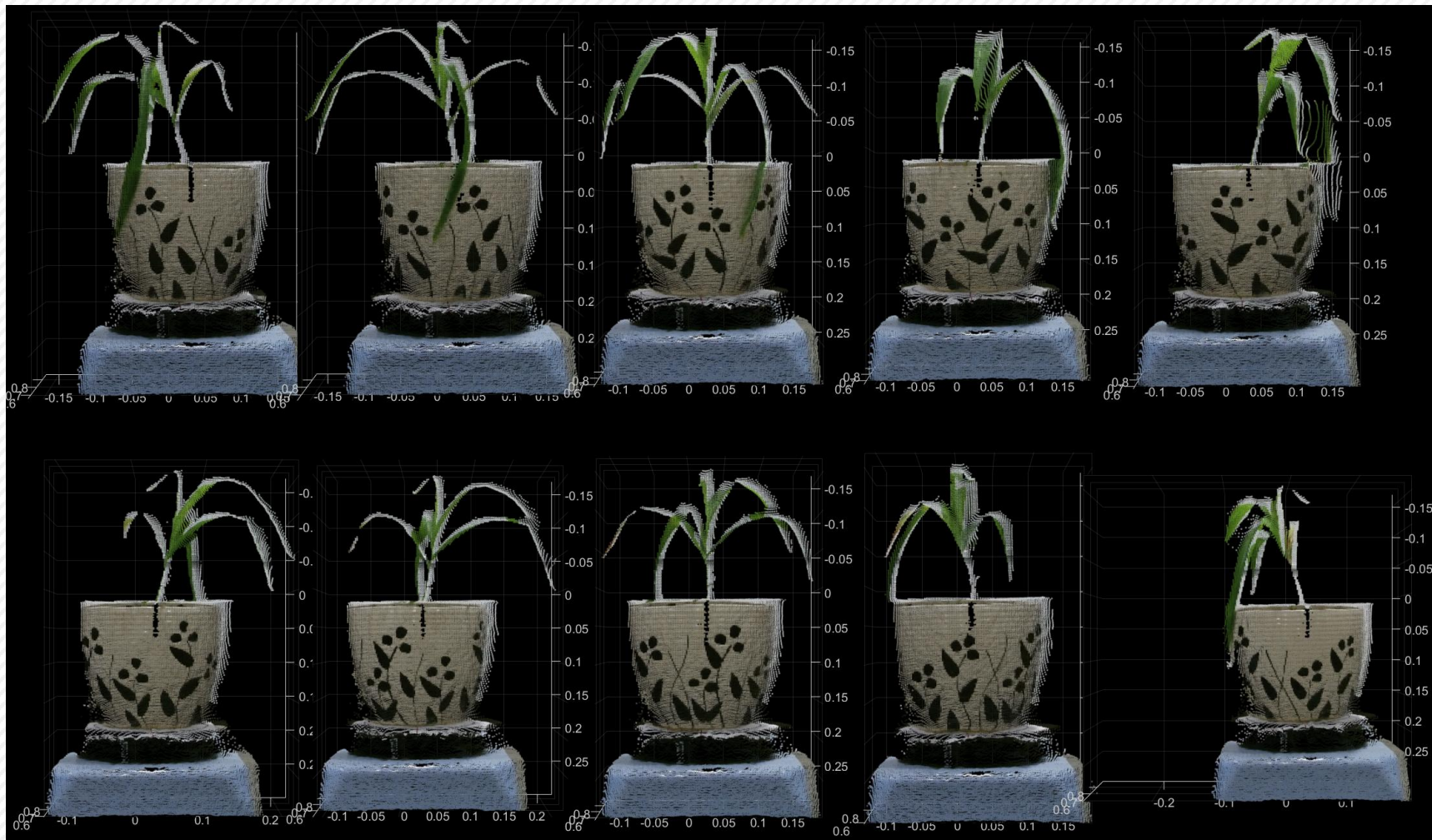
```
% === 自适应设置关键高度 ===  
if i == use_frames(1)  
    % 自适应计算关键高度参数  
    y_range = global_y_max - global_y_min;  
  
    % 截圆高度：位于点云中上部  
    slice_height = global_y_min + 0.65 * y_range;  
  
    % 补色上界：位于点云下部  
    ybuffe = global_y_min + 0.4 * y_range;  
  
    % 几何增强上界：位于点云中上部  
    geometry_interp_ymax = global_y_min + 0.4 * y_range;  
  
    % ICP高度阈值：位于点云上部  
    height_thresh = global_y_min + 0.35 * y_range;
```



03 基于MATLAB/Kinect的表征三维建模

6. 点云及图像数据处理

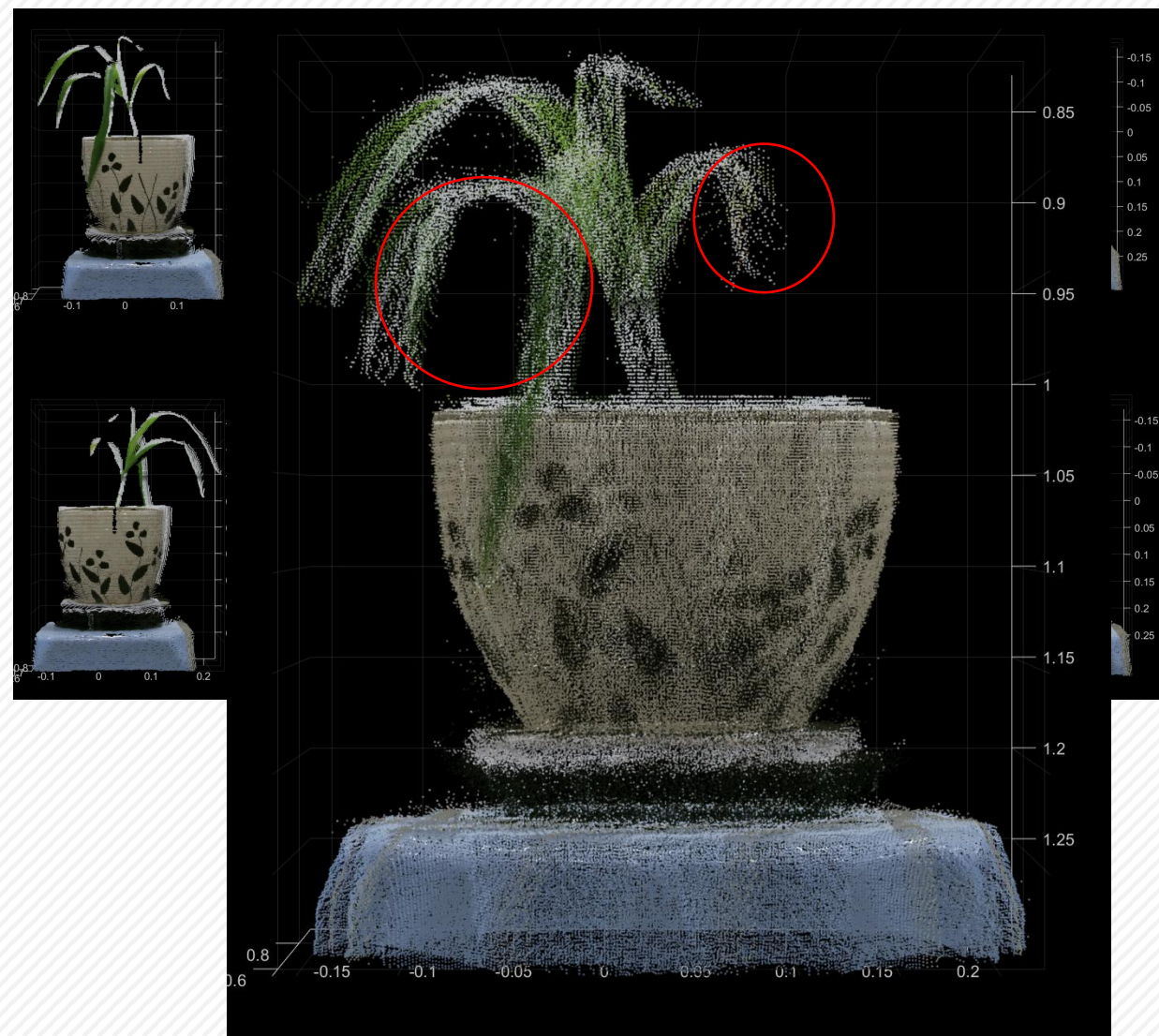
直通滤波去除背景点噪声



03 基于MATLAB/Kinect的表征三维建模

6. 点云及图像数据处理

统计滤波去除玉米叶片周围噪声点

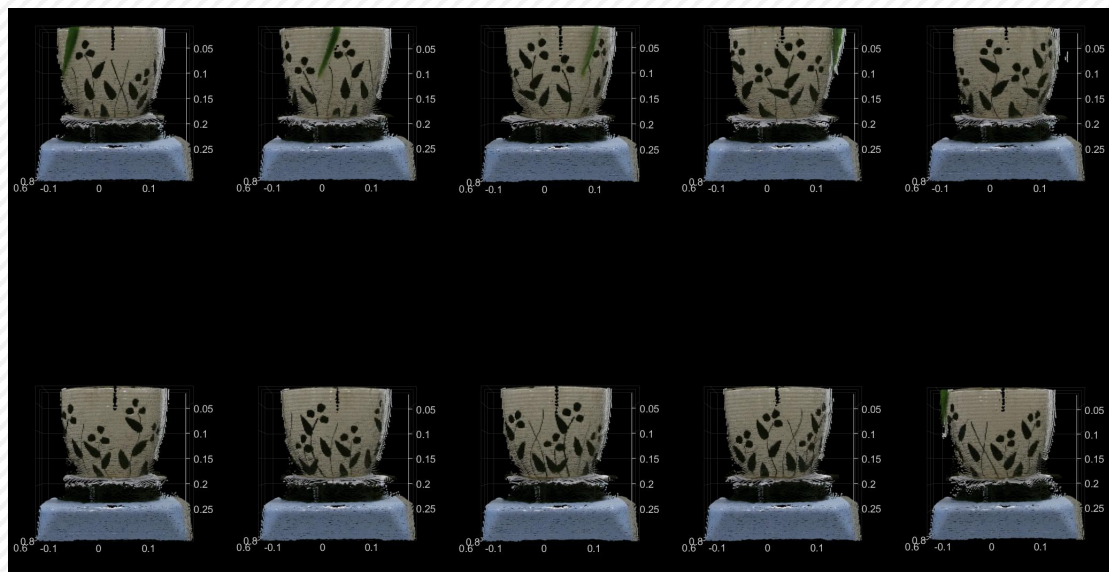


03 基于MATLAB/Kinect的表征三维建模

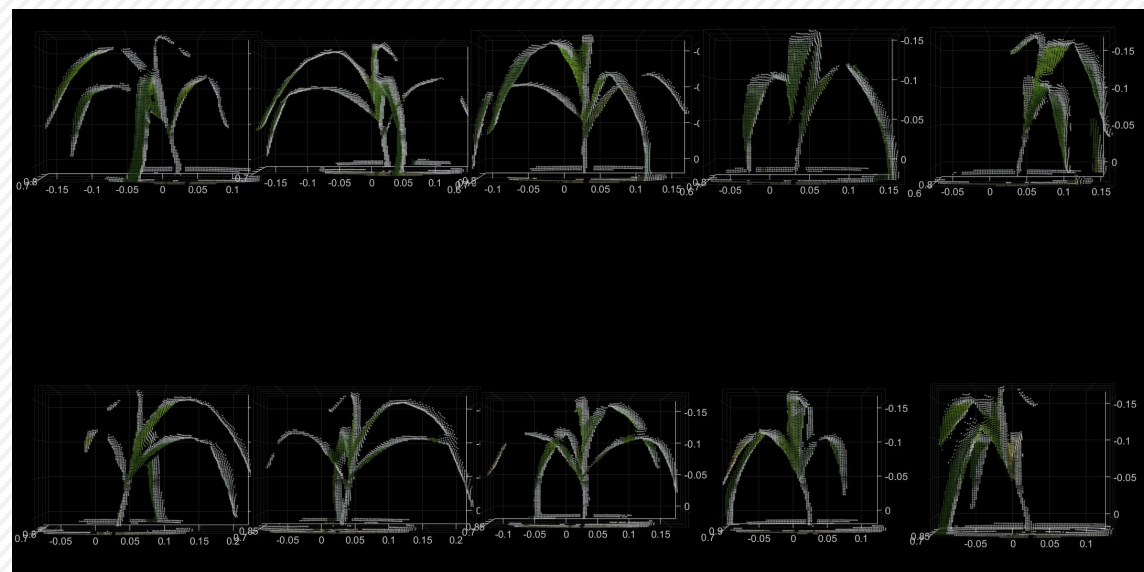
7. 花盆及玉米植株点云匹配

1. 花盆点云与玉米苗点云分离

首先保证各个点云在y轴方向上（即玉米苗和花盆的竖直方向上）处在一个相同的范围内，这里所有点云都近似属于 $[-0.17, 0.30]$ 该范围内，根据任意一点云数据图可以得到玉米苗和花盆在该方向上的大致临界面为 $y=0.02$ ，分别获取 $y>0.02$ 与 $y<0.02$ 的点云坐标并保存即可完成花盆点云与玉米点云的分离



花盆点云



玉米苗点云

03 基于MATLAB/Kinect的表征三维建模

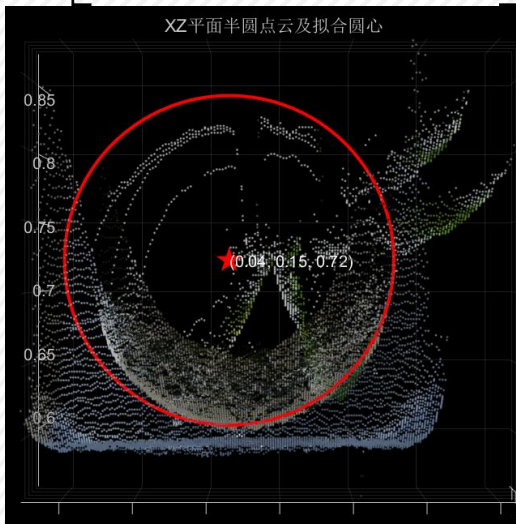
7. 花盆及玉米植株点云匹配

2. 花盆点云重合

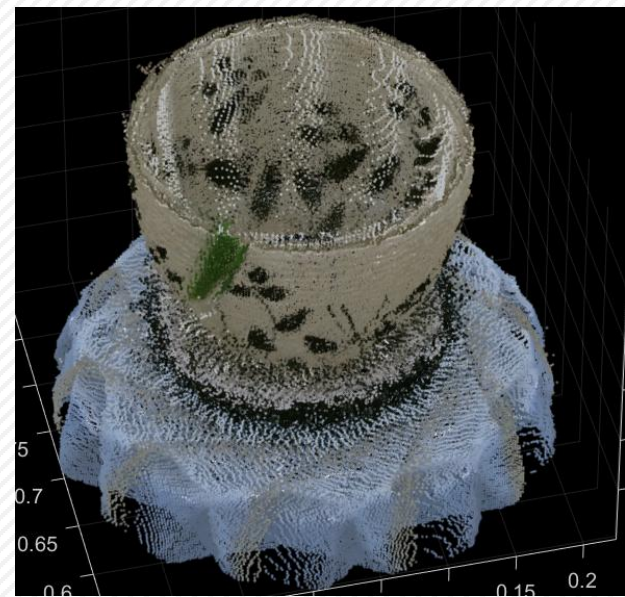
①由于花盆是一个圆台，这里选择将每一份的花盆点云进行圆的拟合获取花盆的圆心坐标；首先获得第一个参考点云圆心坐标 (x_0, y_0, z_0) ，将其作为参考点圆心，再获得其余花盆点云圆心坐标 (x_i, y_i, z_i) ($i=1, \dots, 9$)，对第 i 个花盆点云构造平移矩阵 T (在 y 方向上不需要平移)，每一组花盆点云坐标均为一个 $N \times 3$ 的矩阵 A ，对其添加一列全为1的列，使其成为 $N \times 4$ 的矩阵，然后对其转置得到 A^T ，使用 $T * A^T$ ，获得每一组点云数据平移后的坐标

②获得了每一组点云数据平移后的坐标后，根据每一个点云数据旋转的角度值 $\theta = (i * 36^\circ)$ 进行构造旋转矩阵，利用旋转矩阵 $R * A^T$ ，旋转后将所有点云坐标汇总，获得花盆点云数据 M

$$T = \begin{bmatrix} 1 & 0 & 0 & x_0 - x_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z_0 - z_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$R = \begin{bmatrix} \cos \theta & 0 & \sin \theta & x_0(1 - \cos \theta) - z_0 \sin \theta \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & x_0 \sin \theta + z_0(1 - \cos \theta) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

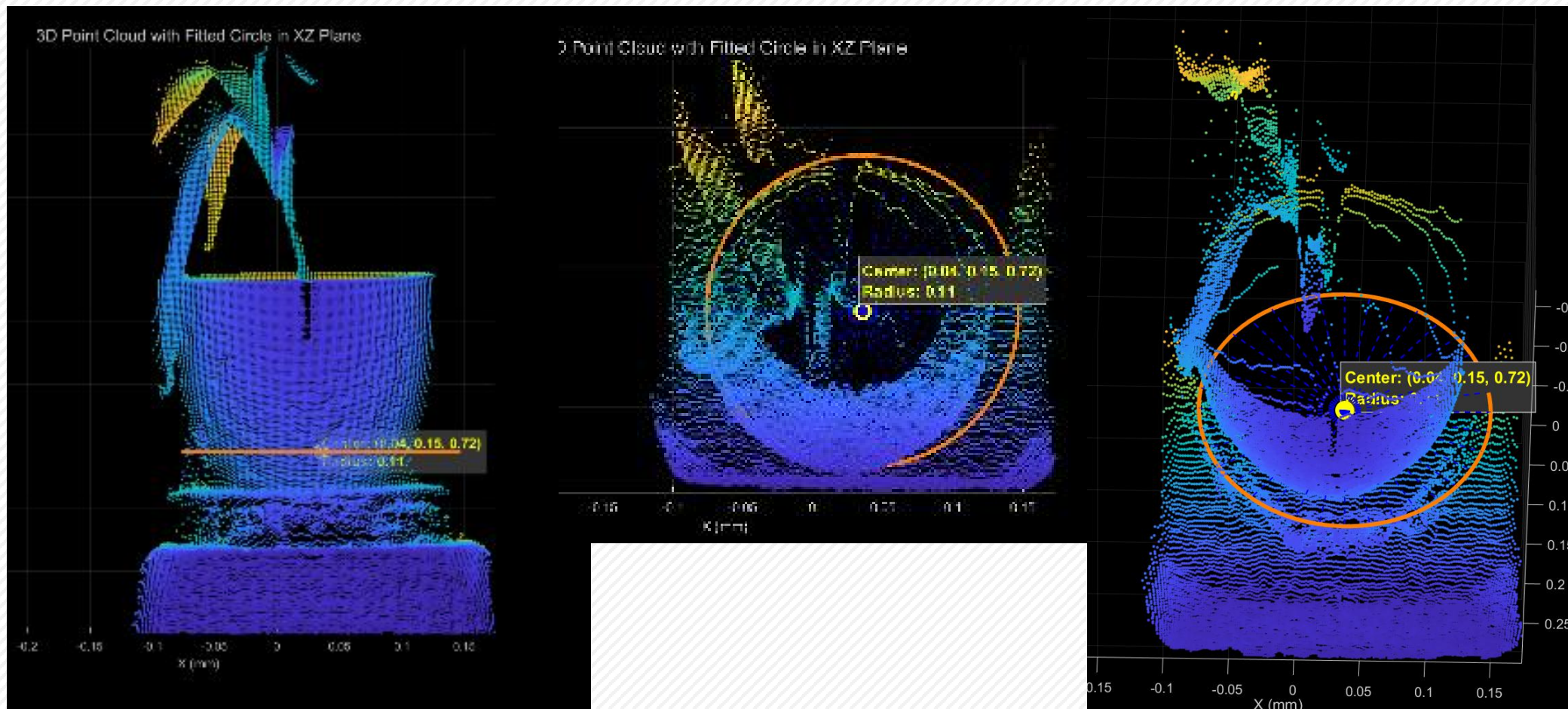


03 基于MATLAB/Kinect的表征三维建模

7. 花盆及玉米植株点云匹配

旋转中心确定

在 XZ 平面对三维点云数据进行二维圆拟合，依据拟合结果精准确定旋转中心，为后续点云整合提供关键定位参考，保证花盆点云在旋转拼接过程中的准确性与一致性。

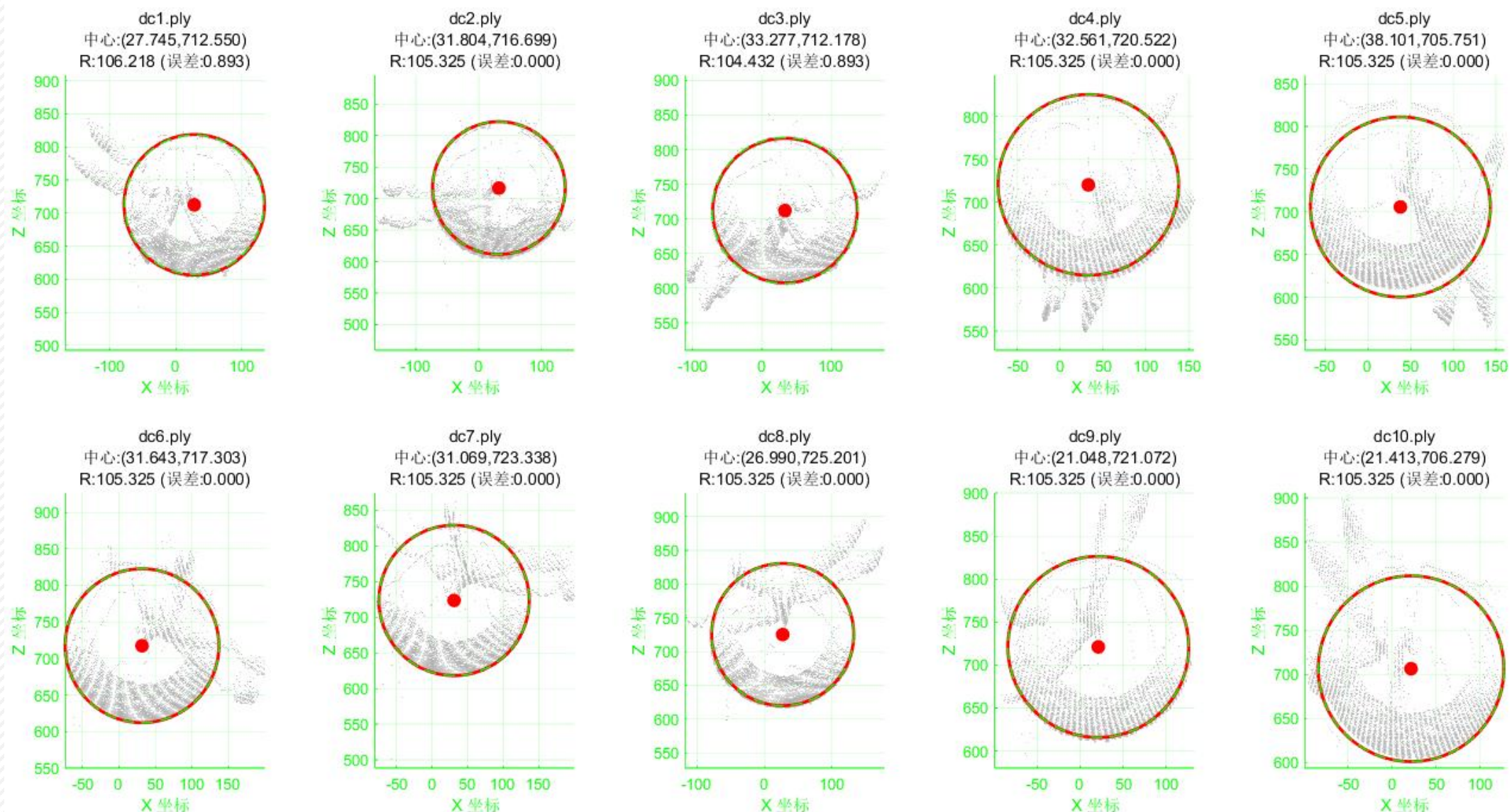


03 基于MATLAB/Kinect的表征三维建模

拟合圆

- ①以圆形点云最外层为边界，
- ②每个直线距离上每个点距离最近、且十组数据拟合出圆直径误差不大于1mm为条件
- ③圆形为旋转轴，旋转轴垂直于XOZ平面，将其保存在单独数组内。

得出圆半径
在:105.325mm

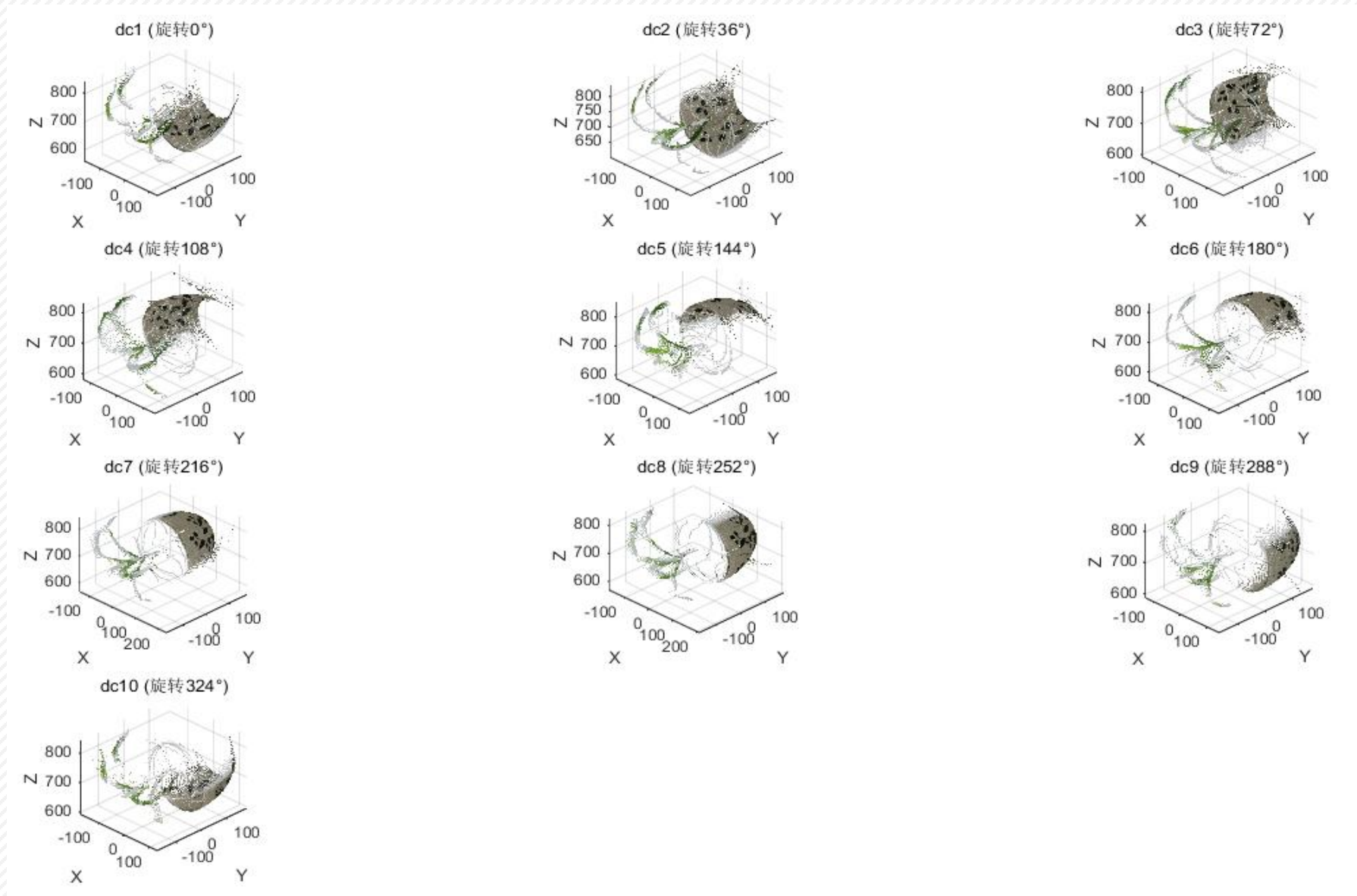


03 基于MATLAB/Kinect的表征三维建模

多视角点云 图像融合

当旋转轴确立后，要将每个点云图像以第一组数据为基准，每组旋转 36° 为进行视角匹配。

得出圆半径在:**105.325mm**



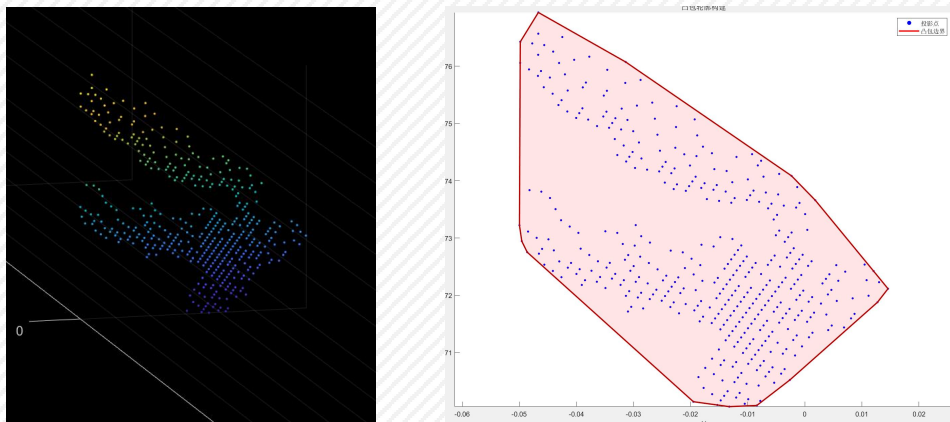
03 基于MATLAB/Kinect的表征三维建模

2.玉米苗点云重合

若采取与花盆点云重合相同的操作，会得到如图结果

①根据每一个点云数据旋转的角度值 $\theta = (i * 36^\circ)$ 进行构造旋转矩阵，利用旋转矩阵 $R * A^T$ ，将十组点云数据进行旋转

②截取十组玉米苗点云数据玉米茎秆范围内的点云，根据任一组玉米苗点云可选取范围 $x[-0.05, 0.05]$ ， $z[0.7, 0.76]$ 的点并将其投影到XZ底面上



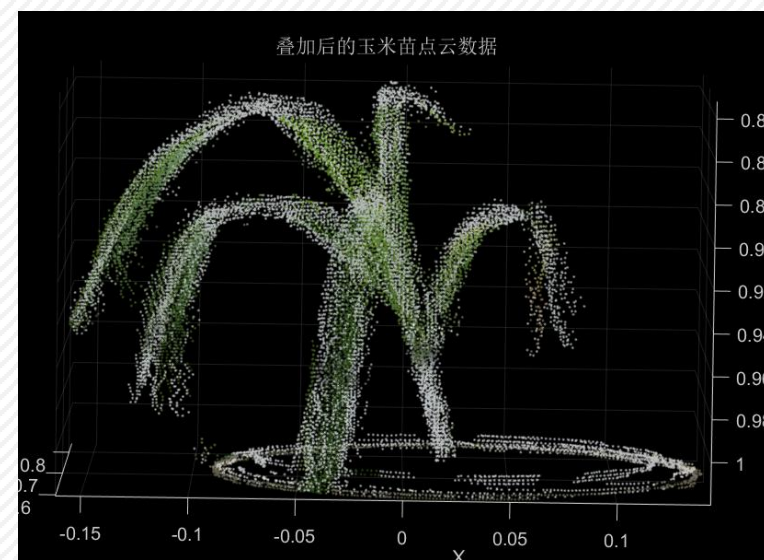
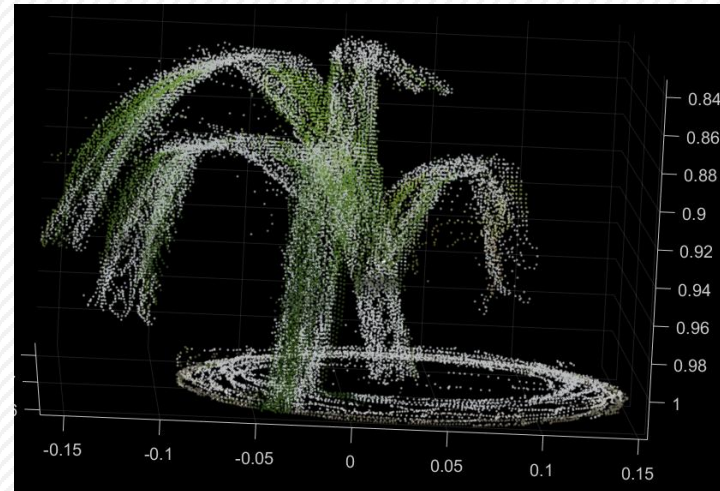
③投影之后使用凸包框选中所有投影点，并根据凸包的边界计算出凸包的中心点位，中心点center为

$$\text{center_x} = (x_{\min} + x_{\max}) / 2$$

$$\text{center_z} = (z_{\min} + z_{\max}) / 2$$

④选择第一个玉米苗茎秆投影后的中心点为参考中心点 (x_0, z_0) ，再获得其余玉米苗茎秆投影后的中心点 (x_i, z_i) ($i=1, \dots, 9$)，对第 i 个玉米苗点云构造平移矩阵 T (在 y 方向上不需要平移)，每一组玉米苗点云坐标均为一个 $N * 3$ 的矩阵 A ，对其添加一列全为1的列，使其成为 $N * 4$ 的矩阵，然后对其转置得到 A^T ，使用 $T * A^T$ ，获得每一组点云数据平移后的坐标

$$T = \begin{bmatrix} 1 & 0 & 0 & x_0 - x_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z_0 - z_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



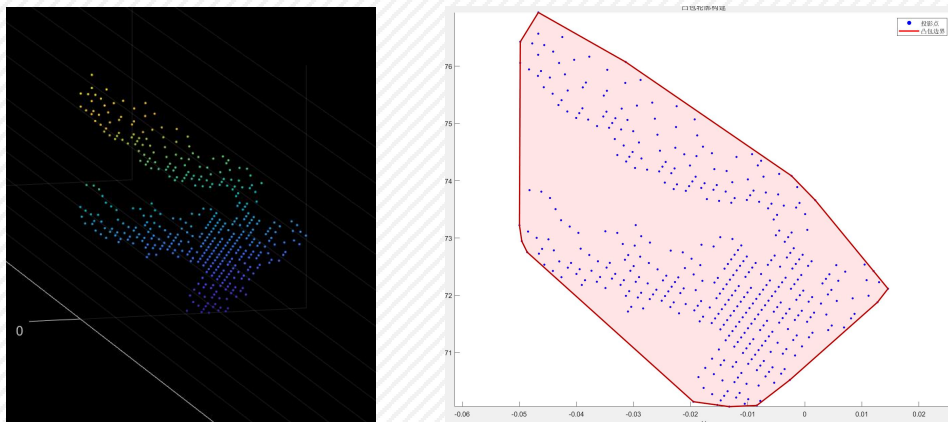
03 基于MATLAB/Kinect的表征三维建模

2.玉米苗点云重合

若采取与花盆点云重合相同的操作，会得到如图结果

①根据每一个点云数据旋转的角度值 $\theta = (i * 36^\circ)$ 进行构造旋转矩阵，利用旋转矩阵 $R * A^T$ ，将十组点云数据进行旋转

②截取十组玉米苗点云数据玉米茎秆范围内的点云，根据任一组玉米苗点云可选取范围 $x[-0.05, 0.05]$ ， $z[0.7, 0.76]$ 的点并将其投影到XZ底面上



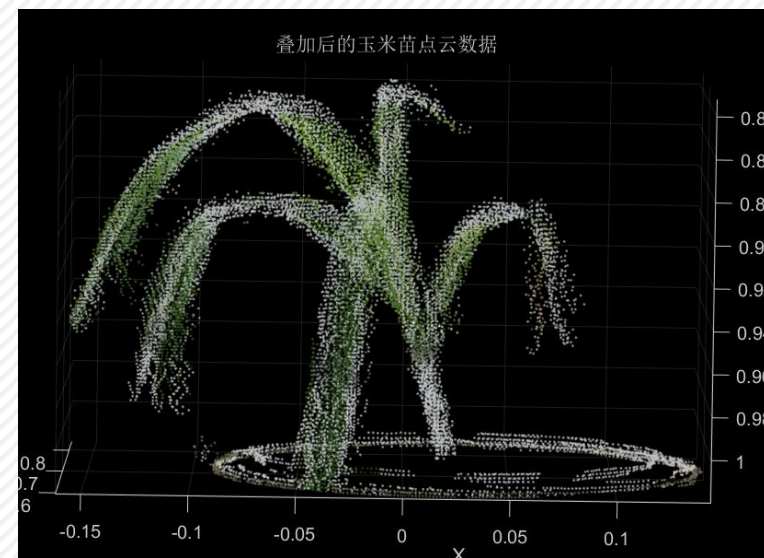
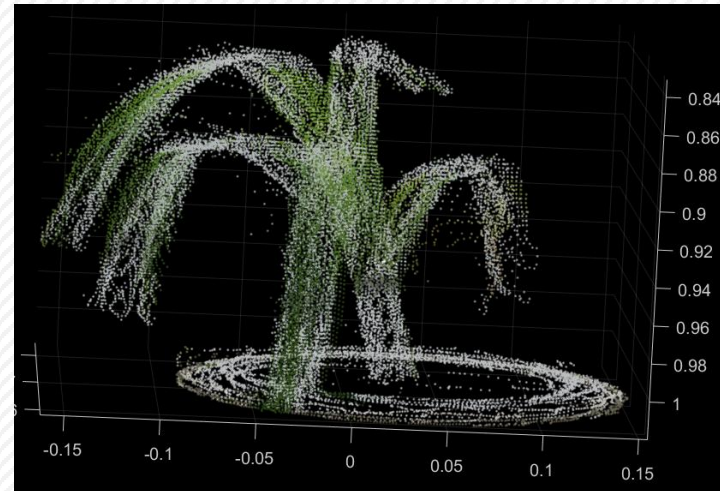
③投影之后使用凸包框选中所有投影点，并根据凸包的边界计算出凸包的中心点位，中心点center为

$$\text{center_x} = (x_{\min} + x_{\max}) / 2$$

$$\text{center_z} = (z_{\min} + z_{\max}) / 2$$

④选择第一个玉米苗茎秆投影后的中心点为参考中心点 (x_0, z_0) ，再获得其余玉米苗茎秆投影后的中心点 (x_i, z_i) ($i=1, \dots, 9$)，对第 i 个玉米苗点云构造平移矩阵 T (在 y 方向上不需要平移)，每一组玉米苗点云坐标均为一个 $N * 3$ 的矩阵 A ，对其添加一列全为1的列，使其成为 $N * 4$ 的矩阵，然后对其转置得到 A^T ，使用 $T * A^T$ ，获得每一组点云数据平移后的坐标

$$T = \begin{bmatrix} 1 & 0 & 0 & x_0 - x_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z_0 - z_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

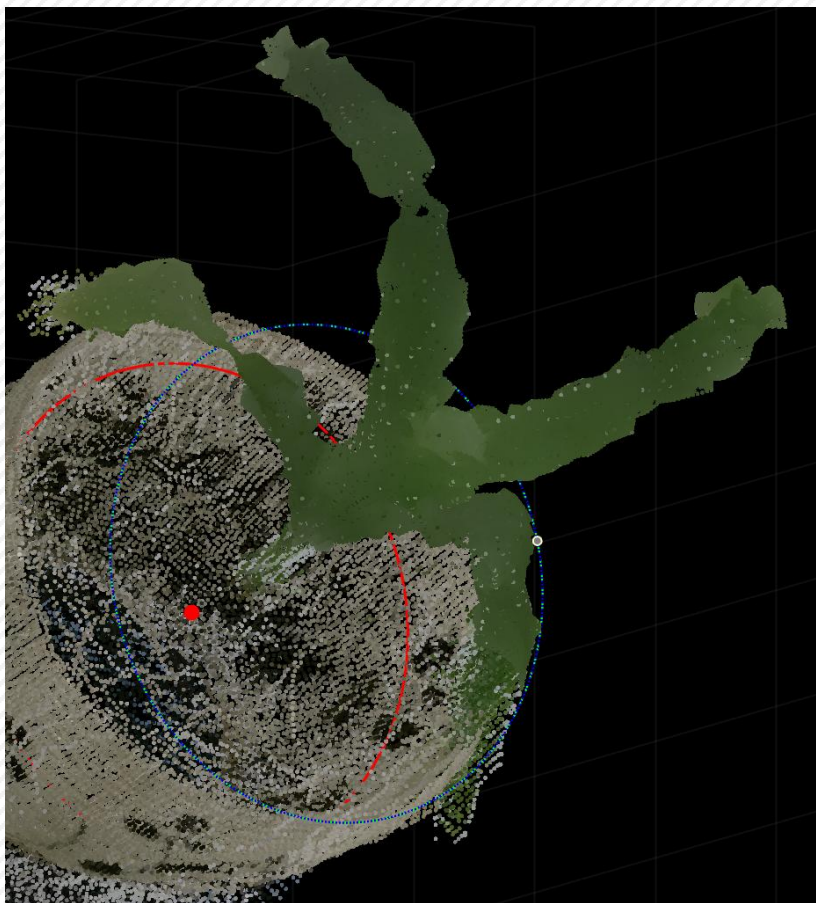


03 基于MATLAB/Kinect的表征三维建模

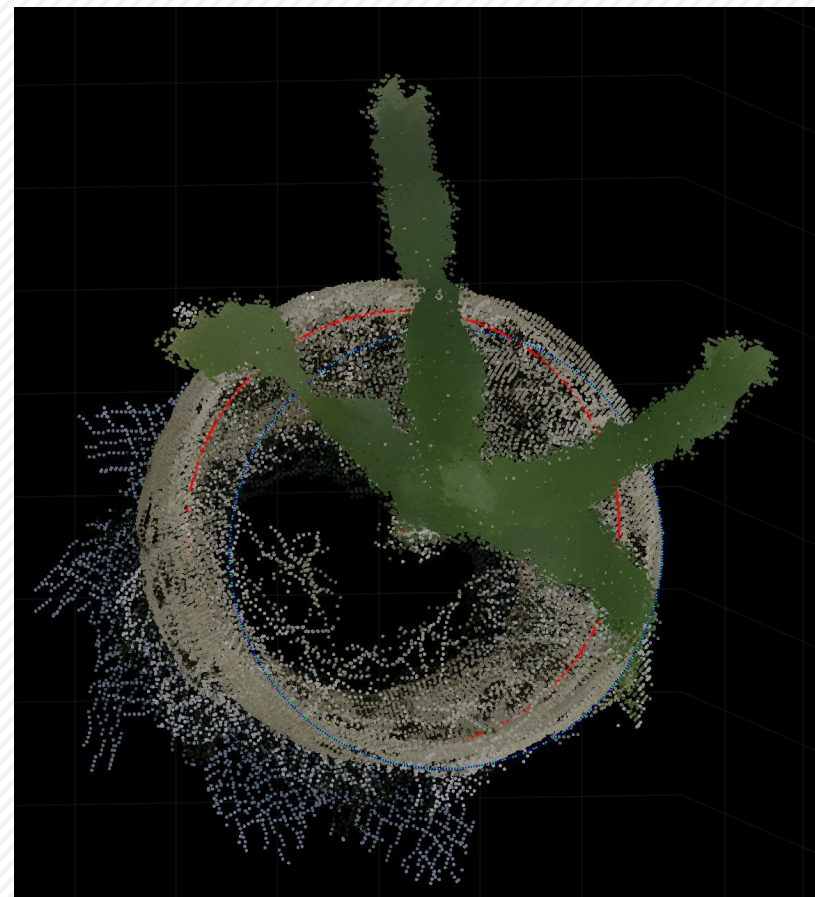
2.玉米苗点云重合

2.体素填缝

- 体素化当前点云：将所有点云映射至体素网格，通过 $\text{round}(\text{xyz}/\text{voxelSize})$ 离散化点位置。
- 枚举 6 邻域体素：对每个已占据体素，检测其六个方向的邻接体素是否为空。
- 填补空体素中心：若某方向为空，则在当前体素与邻体素中心之间添加一个插值新点。
- 颜色补插：通过已有彩色点构建 KD 树，对新点进行反距离加权颜色插值。



几何增强



几何增强+体素增强

色彩缺失的原因

由于拍摄背景采用白板，在深度图像与彩色图像对齐过程中易产生误差，部分植株原始色彩被白板色彩覆盖。

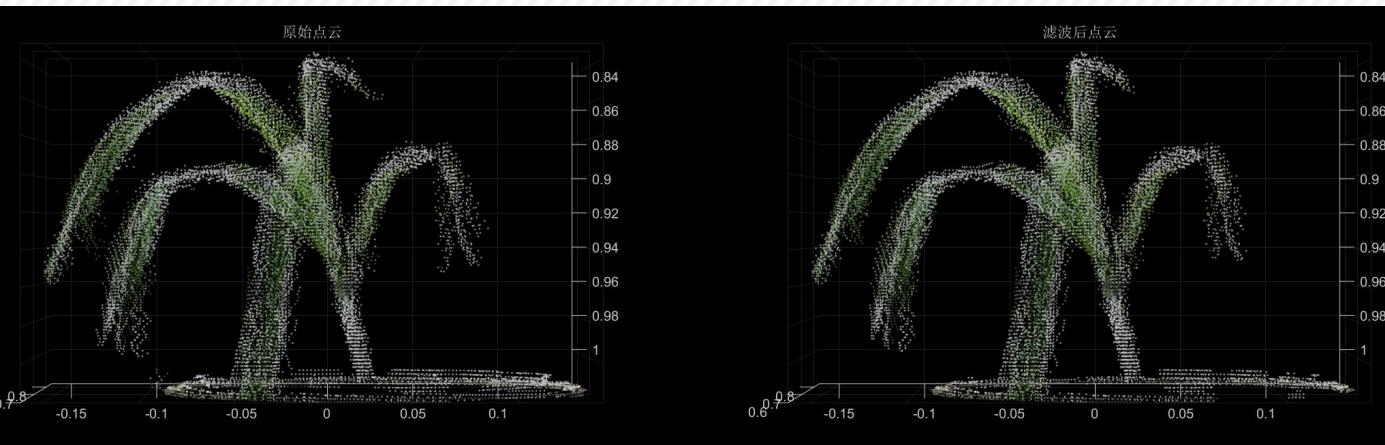
修补思路

对玉米植株点云进行颜色分类与填充。借助 HSV 颜色空间分析，精准识别白色 / 灰色区域（白板色彩），运用邻域传播算法，将其填充为植物特征颜色（绿色 / 黄色），有效增强植物区域的视觉辨识度，使模型色彩更加符合实际植株特征，提升模型视觉效果与观赏性，同时也有助于后续基于色彩的分析与研究工作。

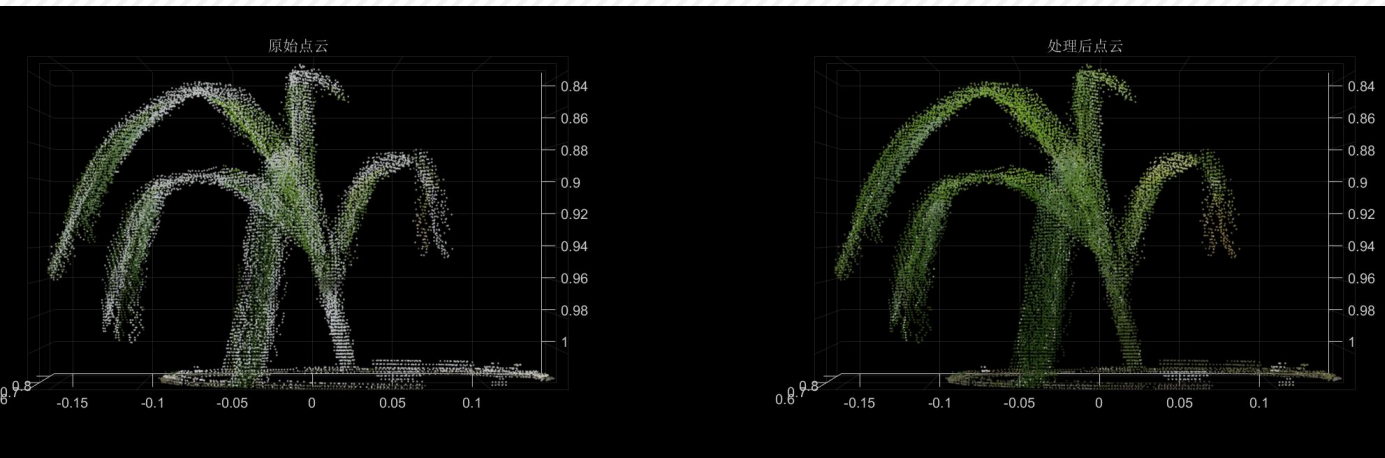


03 基于MATLAB/Kinect的表征三维建模

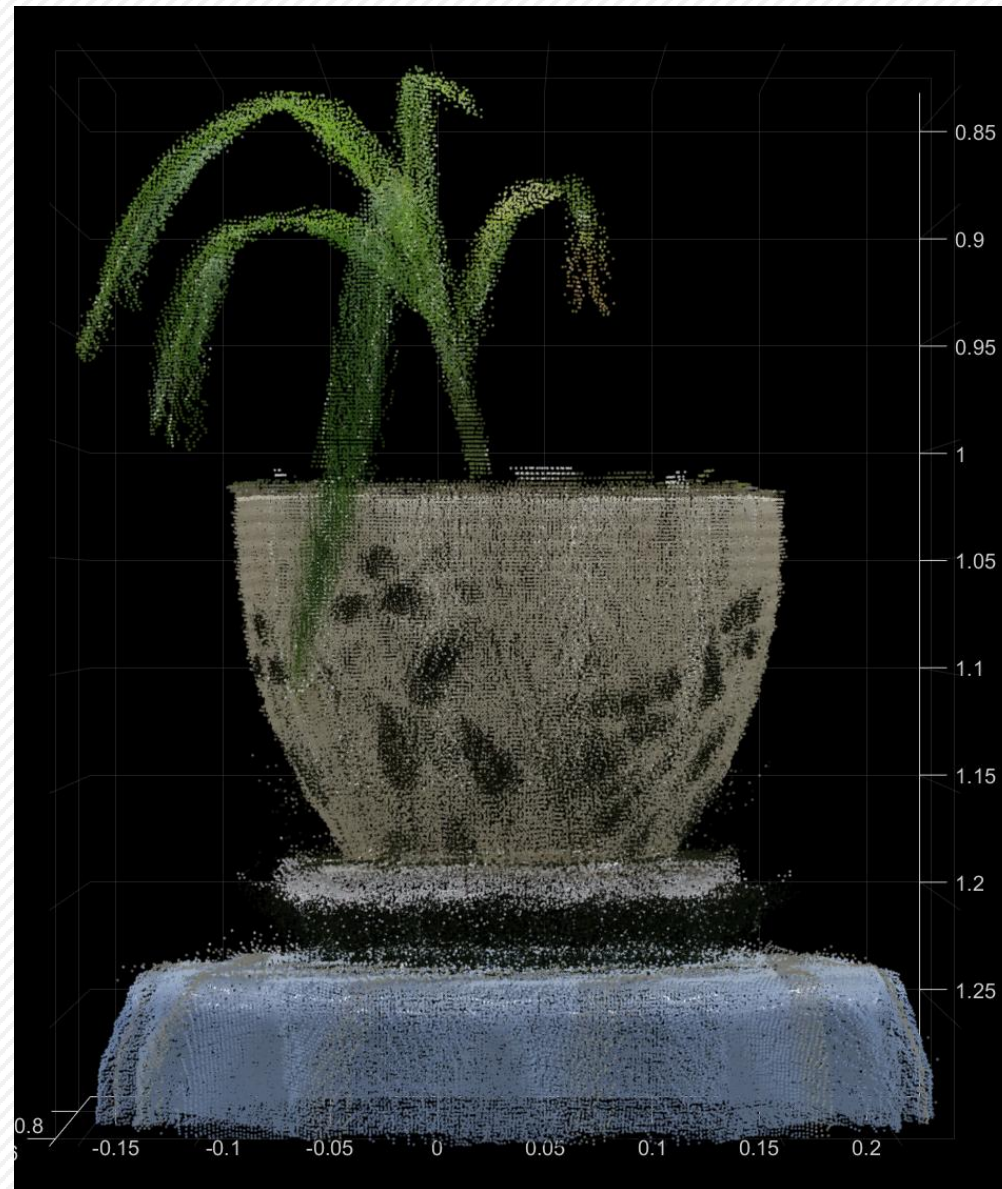
①再次进行统计滤波



②根据周围点云进行颜色赋值，检索点云中带有偏绿色，偏黄色的点云数据作为中心点，并检测其周围点云是否为偏白色或偏灰色点，将其根据该中心点进行重新附色

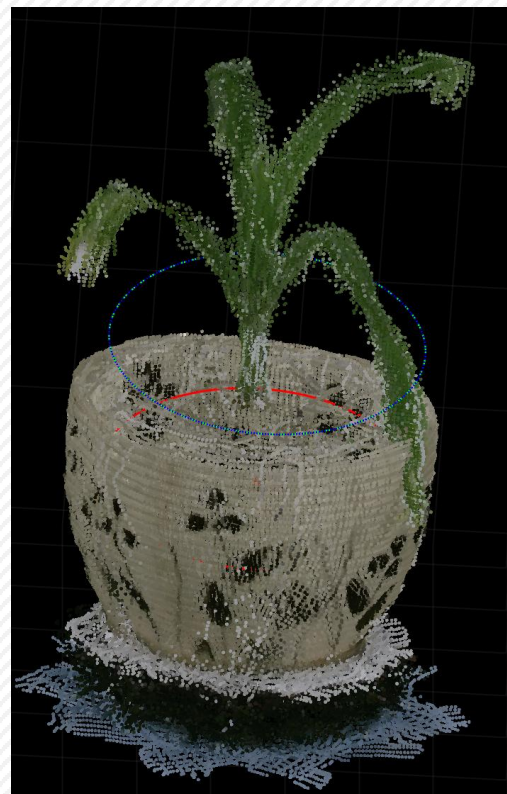
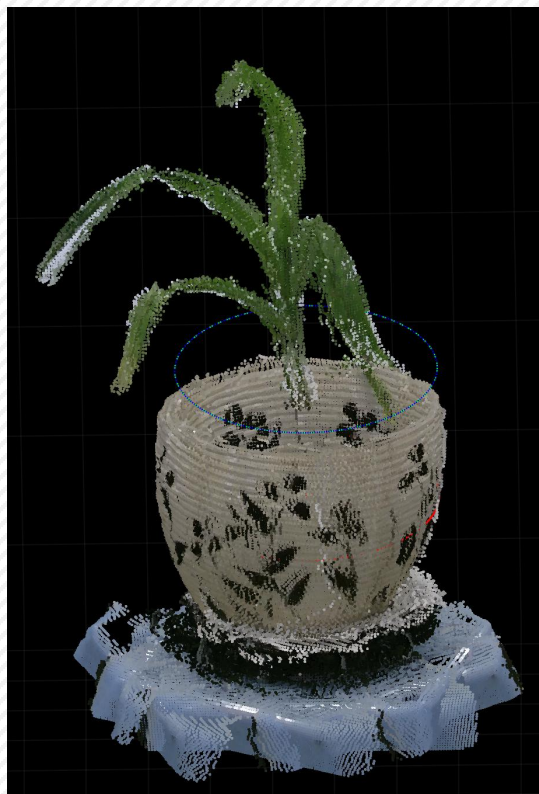
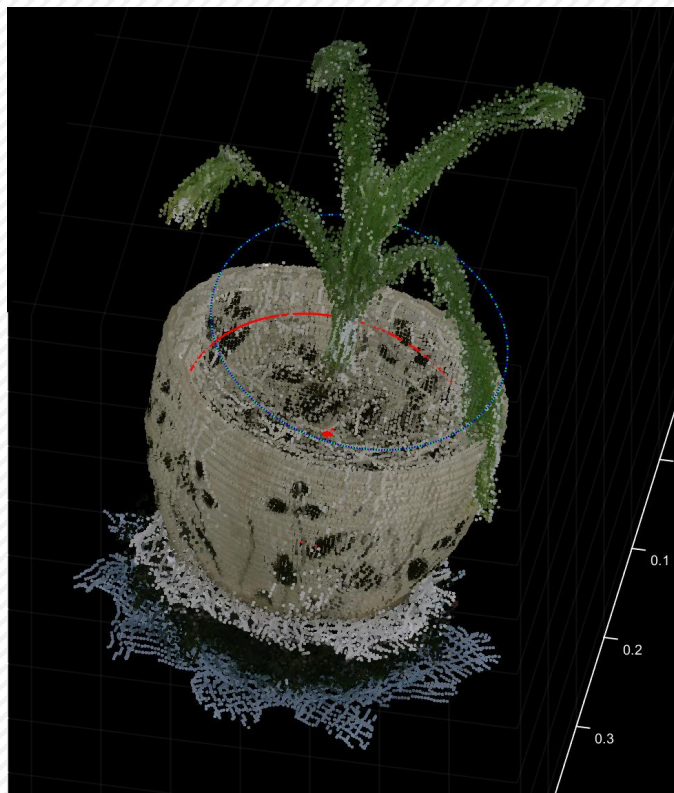


③将花盆点云与玉米苗点云拼接



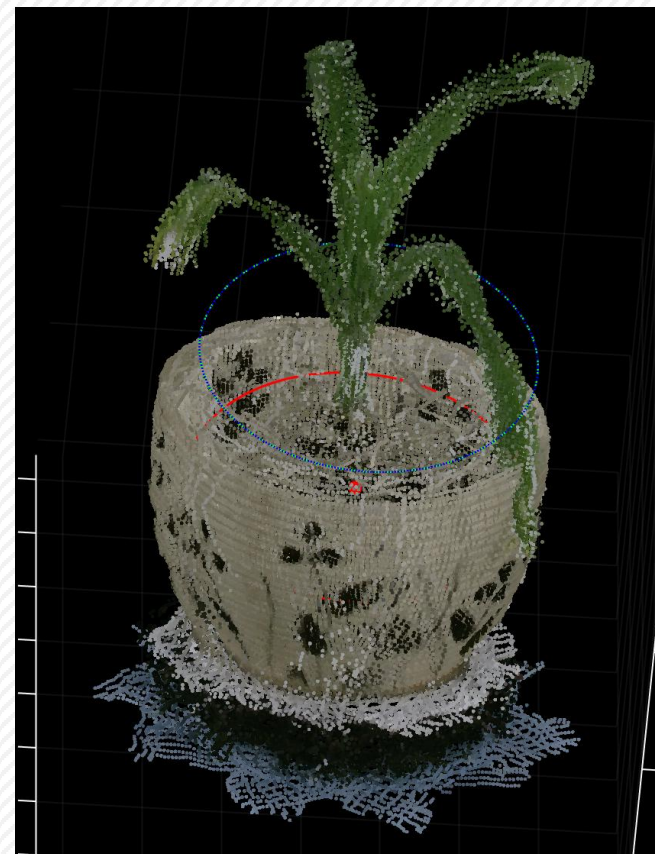
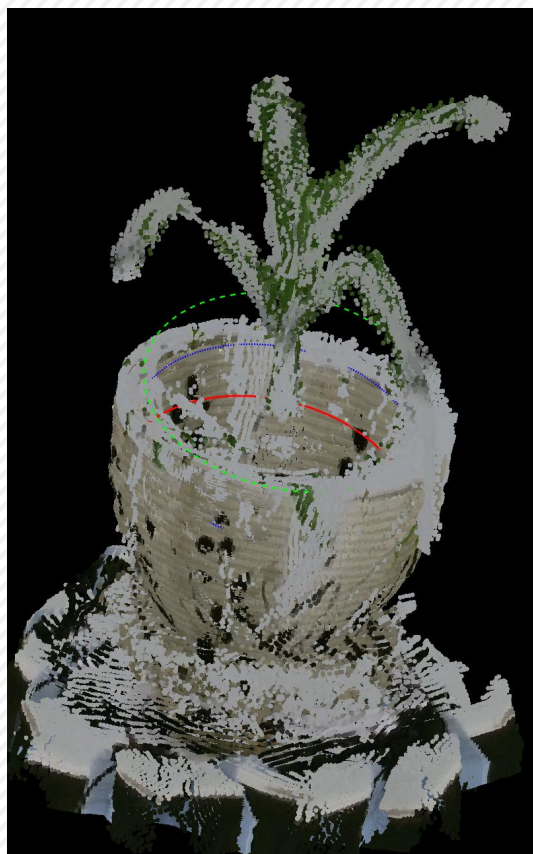
03 基于MATLAB/Kinect的表征三维建模

鲁棒性强：旋转角度、旋转中心、截取高度自动确定，可以适应多组数据



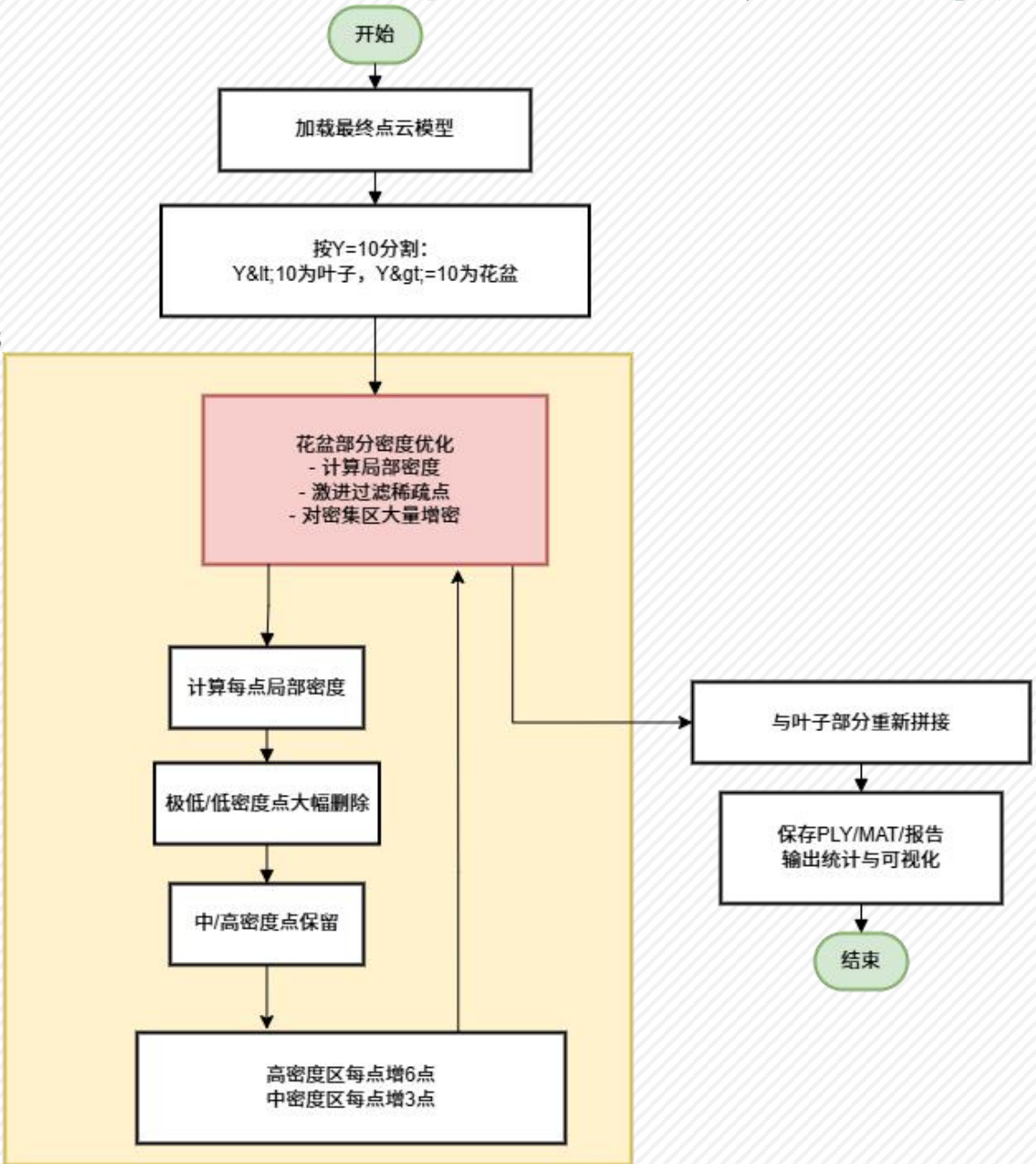
03 基于MATLAB/Kinect的表征三维建模

质量高：不会出现花盆上有多处叶子，颜色还原度高



03 基于MATLAB/Kinect的表征三维建模

花盆部分密度优化



本项目围绕盆栽点云数据处理，设计并实现了从点云去噪、分割、颜色优化、结构识别、密度增强到最终模型拼接与导出的一整套自动化流程。

通过分步聚类、密度分析与高密度补充等算法，有效提升了点云模型的结构清晰度和空间均匀性。最终输出的PLY模型数据不仅更具观感和分析价值。

后续可往优化拍摄道具方面改善殿云三维重建效果，例：参照物、标定物等。



谢谢

THANK YOU!



西南大学工程技术学院

绿色智能车辆与机电系统实验室

欢迎莅临指导交流

冀杰：

Tel: 13594188175

E-mail: 59017045@qq.com

WeChat: maple_swu